

RESEARCH ARTICLE

Predicting Vulnerable Code Changes Using Technical Debt Indicators

RUSEN AKKUS HALEPMOLLASI^{ID 1,2} AND AYSE TOSUN^{ID 1}¹Faculty of Computer and Informatics Engineering, Istanbul Technical University, 34467 Istanbul, Türkiye²TÜBİTAK Informatics and Information Security Research Center, Kocaeli, Türkiye

Corresponding author: Rusen Akkus Halepmollasi (halepmollasi@itu.edu.tr)

ABSTRACT Technical debt refers to the cost of delayed technical work and poor design choices in software systems. Software security focuses on protecting systems against vulnerabilities and exploits. In this study, we explore the relationship between technical debt and software vulnerabilities to understand how technical debt impacts software security. We collected real world data from six software projects, including code changes and corresponding vulnerabilities. We investigate various technical debt indicators, such as code smells, bugs, and software metrics, along with technical debt mitigation methods like refactoring activities, to predict vulnerabilities. The findings reveal a significant association between technical debt and vulnerabilities, highlighting the impact of technical debt on software security. We also identify specific artifacts, such as code smells, that contribute to the emergence of vulnerabilities. By understanding this relationship, software practitioners and project managers can gain insights into the risks associated with accumulated technical debt and take appropriate measures to enhance software security and quality.

INDEX TERMS Code smells, software metrics, technical debt, vulnerability prediction.

I. INTRODUCTION

Software quality can be examined in two dimensions [1]: The first is the absence/existence of bugs or vulnerabilities in the software system, including missing or misinterpreted requirements, design flaws, incomplete functional logic, absent validity, and coding faults. The second is the extent to which Technical Debt (TD) could be managed by complying with various quality attributes. Ozkaya [1] states that assuring quality is based on adhesion to product standards and procedures and a systematic attitude to quality assessment.

TD is a concept that was coined by Cunningham to elucidate the necessity of refactoring to nontechnical software product stakeholders by inspiring from financial debt [2]. TD refers to the cost of delayed technical work related to poor solutions in software systems developed by compromising long-term quality to accomplish short-term objectives [3]. Similar to financial debt, TD can have interest payments that are the extra effort for future development as a result of hasty design decisions and code implementation. Kruchten et al. [4] further categorize TD into visible components,

such as extra functionality and poor external quality, and invisible components, such as documentation debt and code complexity. When TD is left invisible and not repaid, that is, cannot be managed, it can be accumulated and makes maintainability and evolvability difficult. This can lead to software security risks when it is not addressed and not minimized [5]. To make matters worse, the ability to pay “interest” on accumulated debts can be exceeded.

Software security is a quality characteristic among many [6], and refers to the protection of systems and networks against vulnerabilities and exploits by building secure software [7]. To be able to develop secure software, security risks must be identified and mitigated sufficiently early by integrating best security practices into the overall software development life cycle [8], [9]. Harris and Maymi [10] state that providing functional and secure development together offers protection at all layers of software. Thus, in order to reduce the possibility of vulnerabilities, incorporating security-oriented thinking into the systems is a better strategy. Vulnerabilities, less frequently observed but harder to fix compared to other defects, are defects or weaknesses in systems and occur when exploited by a threat actor attempting to perform malicious and unauthorized actions within the system [11].

The associate editor coordinating the review of this manuscript and approving it for publication was Claudia Raibulet^{ID}.

Software systems inevitably have vulnerabilities. However, identifying their causes and addressing them in earlier stages of the life cycle can help developers avoid spending a lot of time and effort on maintenance. Coding and design flaws are reported to often be the causes of vulnerabilities in software projects [12]. The quantification and evaluation of TD becomes more significant in detecting or predicting vulnerabilities. In recent years, many studies have investigated the relationship of TD indicators such as code smells [13], [14] and/or architectural smells [15], [16], bugs [17], and TD mitigation methods [18] with software vulnerabilities. Although some studies, e.g. [15] and [16], argue that there is no significant relationship between architectural smells and software vulnerabilities, others mostly claim that software systems with code smells and bugs have higher security risks, and refactoring bad smells helps improve the security of software systems [14], [17] [19], [20]. Studies have also widely used software metrics, which are indirect TD indicators [17], to analyze the relationship between them and vulnerabilities [21], [22] [23] and to predict vulnerabilities [24], [25] [26], [27]. Therefore, we can claim that TD and software security are related, and in particular, TD in software development can be important indicators of vulnerabilities.

In a recent study, authors examined the competence of various TD indicators to predict security risks at the project level and at class level in software systems [17]. They perform static analysis via SonarQube to extract software metrics, bugs, and code smells as TD indicators and vulnerability density metrics as security indicators. According to their results, TD indicators may be potentially used as software vulnerability indicators as well. However, there is still a gap in the comprehensive prediction of real software vulnerabilities from the perspective of TD. TD indicators can be used more extensively to explain software vulnerabilities in real systems by building prediction models. To our knowledge, very limited studies attempt to predict vulnerabilities using more than one TD indicator.

In this study, our objective is to quantify TD in a larger context and investigate their relationship with real vulnerabilities at the code change level. To our knowledge, this is the first paper that empirically analyzes the relationship of *software vulnerabilities* with *TD direct/indirect indicators*, i.e., code smells, bugs, and software metrics, and *TD mitigation approach*, i.e., refactoring activities, at the code change level using real vulnerability data. We define the following RQs:

- **RQ1:** *How can we predict vulnerable commits with respect to coding artifacts leading to TD?*

By addressing this RQ, the objective is to develop predictive models for vulnerable commits by analyzing certain TD artifacts (e.g., code complexity, code smells, bugs, or refactoring activities) to understand the underlying causes and mechanisms by which TD indicators emerge as vulnerabilities.

- **RQ2:** *How does V-SZZ+ improve vulnerable commit prediction compared to V-SZZ?*

With RQ2, we aim to assess V-SZZ+, our new V-SZZ based labeling approach, which tags vulnerability-inducing commits and all subsequent commits containing vulnerabilities until they are fixed, its effectiveness in terms of prediction performance compared to the state-of-the-art approach, V-SZZ.

- **RQ3:** *How does detecting code smells at every commit perform in identifying vulnerable commits?*

Lastly, we aim to evaluate the effectiveness of our fine-grained, commit-level code smell detection in identifying vulnerable commits against traditional snapshot-based approaches. Our approach analyzes every file modified in each commit, so that all commits are linked to existing or newly introduced code smells of the modified files.

In this context, the main contributions of our study are the following:

- Experiments performed utilizing real-world data are often better representative of industrial software products. Therefore, we collected code changes and corresponding software vulnerabilities from six real-world projects through their GitHub repositories and National Vulnerability Database (NVD)¹ records.
- We used the V-SZZ algorithm to extract the vulnerability inducing commits, instead of SZZ variants, as vulnerabilities are usually introduced in the initial versions of fixing commits, whereas bugs are usually introduced in the recent versions of fixing commits [28]. We have further proposed a new V-SZZ based labeling approach, namely V-SZZ+, that identifies vulnerability-inducing commits and tags all subsequent commits containing vulnerabilities until they are fixed. Our approach provides a more comprehensive labeling of vulnerable commits and enables finer-grained analysis and prediction in the context of TD. Using those, we automatically identified vulnerable code commits, created the first vulnerability data of six different real-world projects from the SmartSHARK dataset, and shared it publicly.
- We have also contributed to SmartSHARK dataset by preparing the first code smell dataset at file and commit level. Unlike the common approach of capturing a few snapshots of the source code, we extracted the code smells from all versions of source files within the projects' version history. We run the PMD tool [29], which is a well-known source code analyzer, on each commit, and separately report all code smells in all file changes. This is a relatively more detailed mapping of commits and code smells, that takes into account the temporal modifications on the files associated to these commits. We believe it helps to eliminate suspicions on consecutive code smells and relate them to vulnerabilities.

¹National Vulnerability Database. <https://nvd.nist.gov>

- Although refactoring activities are among the most common TD repayment methods [30], refactoring activities have never been used to predict vulnerabilities. In this study, we empirically explore the ability of refactoring activities that can mitigate TD to predict vulnerabilities in software products.
- Exploratory studies are typically conducted in the early stages of research when the researcher has limited knowledge of the subject area, while explanatory studies are designed to investigate cause-and-effect relationships between different variables [31]. Previous studies have widely explored mechanisms to detect TD and identify its relationship with software security. In this paper, we performed an explanatory study from a TD perspective using indirect indicators (software metrics), direct indicators (code smells and bugs), and the TD mitigation method (refactoring activities) to predict real software vulnerabilities.
- We also provide a replication package that includes Python code for predictive models, real vulnerabilities, code smells, and other collections extracted from the SmartSHARK dataset and their MongoDB scripts.

The remainder of this paper is structured as follows: Section II summarizes previous works on TD indicators, TD mitigation methods, vulnerability prediction, and the relationship between TD and vulnerabilities by providing a state-of-the-art review. In Section III, we present the data preparation steps. Section IV describes the methodology in which we present used features, built models, and train/test configuration phases to answer each RQ. The obtained results of the study over the considered projects are reported and discussed in Section V, while in Section VI, we analyzed and discussed the threats that could affect the validity of our study. Finally, Section VII concludes the paper.

II. RELATED WORK

We have organized this section into three sub-sections and provide a detailed discussion on TD, vulnerability prediction, and their relationships.

A. TECHNICAL DEBT INDICATORS AND MITIGATION METHODS

There are many studies that report promising ways of identifying, analyzing, and repaying TD. According to a recent survey conducted by Alfayez et al. [32], which analyzed 128,897 TD-related tweets to explore how the popularity of TD has changed over time, as well as how TD is discussed on Twitter, there is an overall rise of interest in TD on Twitter. Also, TD causes and TD repayment are among the most popular categories. Furthermore, in an observational study [33], authors examined how TD term is used on Stack Overflow, Software Engineering, and Project Management Stack Exchange Q&A websites. They state that the set of compiled and analyzed questions can be divided into

14 categories, including TD repayment, TD management, TD identification, etc. [33].

Kruchten et al. [34] claim that in software intensive systems, accumulated TD affects mainly the evolvability and maintainability when it is not addressed and not minimized. Moreover, several studies [35], [36] [37] reveal that TD needs to be actively managed as it continues to accumulate during the system development once occurring. Avgeriou et al. [38] state that TD management, i.e., identification, evaluation, and repayment, is a continuous process, hence, it is important to integrate this process into the overall software development process to ensure that the software is maintainable and evolvable. TD provides a way to measure the current sustainability of a system when managed effectively, nevertheless, it can spiral out of control when not managed properly [38].

Li et al. [30] carried out a systematic mapping study for the comprehension of TD concept. They indicate that TD identification and repayment are the most frequently discussed and explored in TD management activities. Various debt types due to intentional or unintentional technical decisions in software systems can be detected by TD identification activities. The detected TD can be resolved or mitigated using TD repayment activities such as refactoring, without changing the systems' functionality [30].

Paudel et al. [39] conducted an industrial case study to empirically investigate the relationship between TD and software delivery lead time. They analyzed software components using SonarQube and Jira issue data and found that TD accounted for 5% to 41% of the variance in lead time. Their findings highlight the multifactorial nature of delivery performance and emphasize the need for broader investigations beyond TD metrics alone. [39].

Previous studies uncovered various types of TD that can be identified using several indicators. Some research efforts focus mainly on the code-based analysis of the existing debt due to lack of tools that can accurately identify TD. To identify TD, those studies employ static analysis methods (e.g., lines of code, code complexity, duplicate code, etc.) or dynamic analysis methods (e.g., unit test, code coverage, etc.) [40], [41] [42]. Also, some tools such as SonarQube [43], SQUORE [44], and Kiuwan [45] used by development teams are introduced to manage source code quality, including TD. Besides, some studies suggest looking into the low level design of the source code [46], [47], [48] to examine their relationship with TD. Examples of such code analysis include, but are not limited to code smells [49], design smells [50] object oriented design such as coupling and cohesion [51], and modularity violations [52].

Refactoring, which eliminates code smells and prevents the faults occurring, is generally accepted as the most popular TD repayment method [53], [54] [55], [56]. Therefore, in the last two decades, the body of research on refactoring and its relationship with code smells or faults has been overgrowing [55], [57]. Benidris et al. [58] investigated how Technical Debt Density (TDD) evolves across multiple

software releases and how it relates to refactoring activities and highlighted the strategic importance of managing not just the amount but also the intensity of TD throughout software evolution. Observational papers examined the reasons and methods behind developers carrying out refactoring [59], [60] [61], which specific types of refactoring are associated with performance indicators of the application [62], [63], and its relation to developers behaviors [64], [65] [66] and the impact of refactoring on TD indicators such as code smells and faults [56], [67] [68].

Pérez et al. [53] conducted an industrial survey research study with 432 software practitioners to analyze the current state of the practice of TD payment from the software developers' point of view. They present the top five TD repayment activities and their rates regarding the age and size of the systems. Authors also compare the similarity between TD repayment activities for the most cited TD types, i.e., design debt, test debt, and code debt, and observe a significant similarity in the code debt and design debt repayment activities. They report that the most common TD repayment practice used by developers is refactoring [53].

B. VULNERABILITY PREDICTION

Vulnerability prediction has received significant attention from software researchers and practitioners as vulnerabilities may have a greater impact than other defects in terms of maintenance, costs, loss of organizations' reputation, and violation of user privacy. There exist many studies for vulnerability prediction in terms of the source code representations (e.g., text mining techniques or static analysis approach), the dataset (e.g., published vulnerability reports or static analysis), the granularity (e.g., class-file level, function/method level, etc.), or ML/DL algorithms. A recent systematic mapping study provides a comprehensive overview of these efforts, categorizing existing research based on techniques, datasets, granularity levels, and prediction methods [69].

Wang et al. [70] investigated the impact of various DL models on detecting vulnerabilities in Python code. They compared the effectiveness of CNNs, RNNs, and transformers in capturing code semantics for vulnerability prediction. Their results demonstrated that transformer-based models achieve higher accuracy in vulnerability detection compared to traditional DL approaches [70].

Zhang et al. [71] proposed a GNN-based approach for vulnerability detection and introduced counterfactual explanations to improve model interpretability. They emphasized the importance of explainable AI in security-related tasks. According to their results, proposed CFExplainer outperforms state-of-the-art factual reasoning-based explainers [71].

Zhou et al. [72] present Devign, a general GNN based model to identify many types of vulnerabilities by combining extensive code semantic representations. Their approach contains a new Conv module that uses node features of a graph extracted from gated graph recurrent units that are used

to learn the node embeddings. To train the model, they created a manually labelled dataset from four large-scale open-source C projects and used Multi-Layer Perceptron (MLP). They used accuracy and F1-measure to evaluate the performance, and reported that Devign has significantly better performance than the state-of-the-art, with an average of 10.51% higher accuracy and 8.68% higher F1-measure.

Nguyen et al. [73] propose a code metric based on two-level dependency graphs of software systems, i.e., component dependency graphs and member dependency graphs, in predicting vulnerable components. The dependencies were based on the relationship among software granularities, including components, classes, functions, and variables that were extracted from static code analyzers and were stated according to the nodes' and edges' attributes. They built ML models with Bayesian Network, Neural Networks, Naive Bayes, SMO, and Random Forest using those attributes. The precision is reported as 54% when they reach the highest recall rate of 75% using Bayesian Network model, and the recall was 39% when they reach the highest precision rate of 85% using SMO model.

Yamaguchi et al. [74] propose a graph based source code representation, namely a code property graph combining AST, program dependency graph, and control flow graph into a joint data structure to model templates for common vulnerabilities with graph traversals. They performed experiments on the source code of the Linux kernel as a case study and identified 18 unknown vulnerabilities. Moreover, they analyzed the source code of Firefox, LibPNG, and Pidgin, etc. in various studies [75], [76] [77].

Sahin et al. [78] also propose a vulnerability prediction model that employed code representations through graph node embeddings and token based embeddings to determine the vulnerability of a function version. Their approach involves representing function versions as node embeddings derived from their Abstract Syntax Tree (AST). Next, they developed models utilizing two Graph Neural Networks with node embeddings, and compared these with Convolutional Neural Network (CNN) and Support Vector Machine (SVM) using token representations. The empirical analysis on real vulnerability data reveals that node embeddings of function versions can be used to detect vulnerability-inducing ones.

In [26], authors created developer profiles according to vulnerabilities and examined the impact of those developer profiles on vulnerability prediction models. They used Heterogeneous Information Network (HIN) analysis to profile the developers' vulnerability inducing/fixing behaviors in software products. Also, they compared developer profiles against software code metrics and evaluated the performance of constructed classifiers in terms of precision, recall, and F1-score. According to their findings, developer profiles are useful to predict vulnerabilities when combined with other metrics.

Recent studies have explored the potential of Large Language Models (LLMs) for software vulnerability detection under various constraints. Le et al. [79] demonstrated that

while CodeBERT struggles with low-resource languages such as Kotlin, Swift, and Rust, ChatGPT offers notable improvements in vulnerability prediction. Lu et al. [80] introduced GRACE, a framework that augments LLMs with graph-structured code representations and in-context learning, and achieved significantly improved performance over state-of-the-art baselines on C/C++ datasets.

C. TECHNICAL DEBT AND VULNERABILITIES

The relationships between vulnerabilities, TD, and defects have become a topic of increasing interest in software engineering as TD indicators and management get more mature [5]. In this context, some studies argue the feasibility of quantifying software security using TD concept. For instance, Silva et al. [81] carried out research to identify the different TD types that can result in the rejection of a pull request and analyzed 1,722 pull requests. Authors classified TD in seven categories, including design debt, performance debt, and security debt, etc. They described security debt as a specific solution that jeopardizes the system's security.

Rindell et al. [22] offer guidelines for extending the TD paradigm to include software security, and state the software security risks in terms of TD. They examined the technical and organizational impacts of implementing the concept of managing security risks as TD in software systems. Authors evaluated security-related TD in terms of identification, measurability, management, and repayment. According to study, addressing security risks and TD together can make managing security risk and addressing underlying quality issues more visible and enable more communication between developers, security professionals, and managers [22]. Furthermore, Rindell and Holvitie [23] offer a two-fold approach to define security debt. They use the term: (i) to identify TD using security validation or verification techniques, (ii) to identify TD in a security-critical software component. They provide a security debt management strategy to handle new TD that can have security issues and to identify, evaluate, and mitigate security debt.

Besides, Izurieta et al. [82] offer a five-step approach for prioritising security related TD items by employing the Quamoco and QATCH quality models, and catalog and scoring systems, namely Common Weakness Enumeration (CWE) and Common Weakness Scoring System (CWSS). In [83], authors extended this approach by simulating attacks and creating dependency graphs between external attacks and the technological repercussions related to CWEs. In line with the conclusions of both studies, they express that security is a specific case as TD related to security can be quantified both in terms of maintainability and in terms of damage to a business, thus, TD needs to be dealt with as soon as possible in relation to vulnerabilities.

There are also studies that report empirical evidence for the relationships between TD, defects, and software security. For example, Nord et al. [21] explored the relationship between software vulnerabilities and TD indicators to

determine whether software components with accrued TD are vulnerability-prone. They used TD based on problematical design decisions to determine hard-to-find vulnerabilities. Authors report their preliminary findings from an investigation of Chromium, including measurement based on the classification of design consequences in issue tracking systems, and the static analysis of the code anomalies such as fault, vulnerability, and design flaw. They identified 79 software vulnerabilities related to 289 source code files containing TD, that is, design flaws were detected. TD is defined according to the methodology presented in [84], in which a design model is constructed as a set of design rule spaces.

Siavvas et al. [19] present an empirical study to explore the relationship between TD and software security over 50 open source Java projects mined from GitHub. They employed static analysis using SonarQube and FindBugs to find out whether TD and software security have a statistically meaningful relationship. When comparing TD and vulnerability density, they calculated them based on SQALE Density and Static Analysis Vulnerability Density (SAVD), respectively. According to their preliminary results, there exists a statistically significant, positive, and strong correlation between TD and the vulnerability densities. Recently, in [17], authors extended the preliminary analysis and mined, both at the project and class level of granularity, 210 open source Java projects to examine whether TD indicators are also indicators of security risks. They calculated TD indicators using SonarQube and quantified security risk in terms of the SAVD metric by discretizing into Security Risk Levels (SRLs). They state that TD indicators might also function as security indicators [17].

III. DATA PREPARATION

One of the most essential and time-consuming phases of building an ML pipeline is data preparation [85]. A recent systematic literature review [86] highlights the challenges and solutions about vulnerability related data preparation and offers data-oriented steps in an ML based workflow. Authors define four steps as data requirements, data collection, data labeling, and data cleaning. In our analysis, we prepared data according to those steps throughout our software vulnerability prediction - data preparation pipeline (Figure 1).

A. DATA REQUIREMENTS

To achieve the desired model accomplishment, we have determined the data requirements as follows:

- We chose to mine projects developed in Java, as it is a widely used programming language, and there are many tools that can extract TD indicators and mitigation methods in code developed in Java language.
- We target detection capabilities for various vulnerability types, rather than a certain vulnerability type. Thus, we included entire vulnerability types in the mined projects in our analysis.

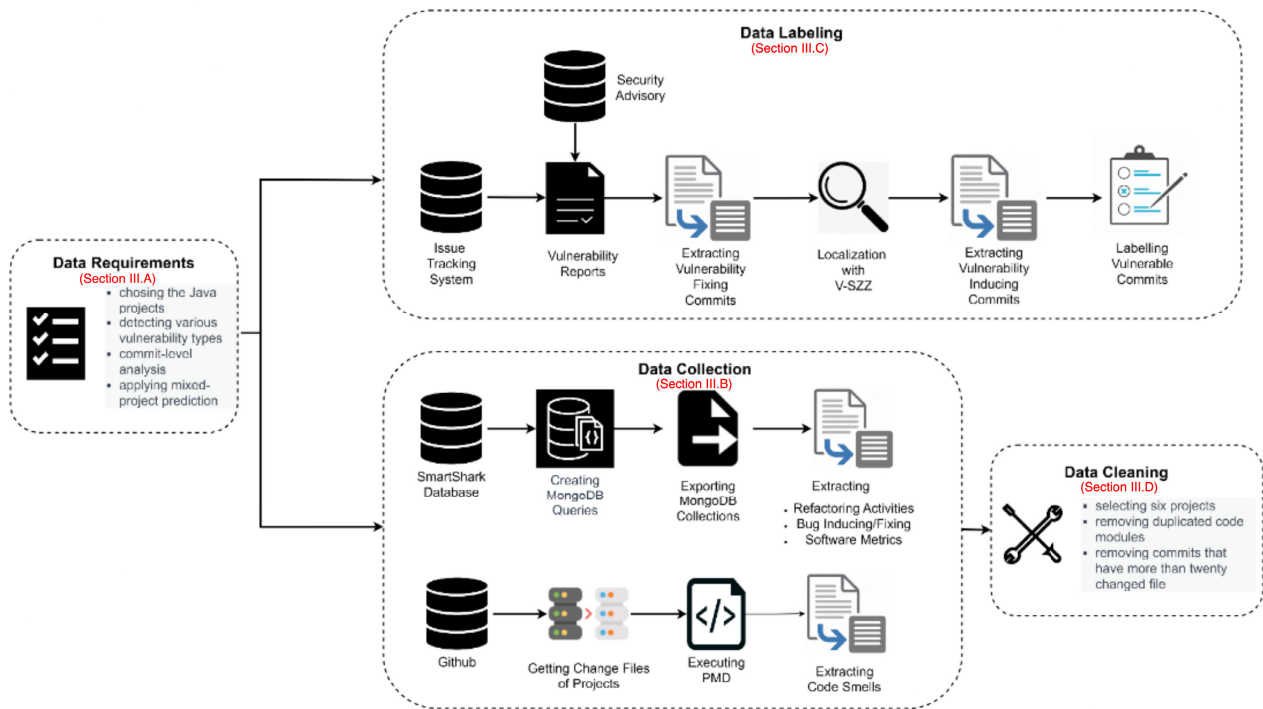


FIGURE 1. Data preparation pipeline to predict the software vulnerabilities. Adapted from [86].

- The commit granularity level is critical to evaluate predictive models. There are different granularity levels, such as component level, file-class level, function-method level, and commit level. We prefer to use commit/change level to be able to analyse code changes and enclose the vulnerable code snippets after each commit.
- We applied mixed-project prediction approach, that is, we merged the data from several projects to obtain enough data for training and testing of the implemented models.

B. DATA COLLECTION

In this study, we used SmartSHARK dataset release 2.1 full version² for extracting refactoring activities, bug fixing/inducing commits, software metrics, and vulnerability-fixing/inducing commits. SmartSHARK has recently been created by Trautsch et al. [87], and contains 77 projects from the well-known ecosystem, namely Apache Software Foundation (ASF) repository.³ The size of the entire dataset is 1.2 TB and is stored in a MongoDB. There are, in total, 47,303 pull requests, 366,322 commits, and 163,057 issues across all projects developed in Java language. Each project includes between 1,000 and 20,000 commits.

SmartSHARK is a very comprehensive dataset and contains various important information about projects in

many MongoDB collections. Since we focus on analysing the relationship between vulnerabilities and TD indicators such as code smells and bugs, indirect indicators such as software metrics, and mitigation methods such as refactoring activities, we selected only the collections related to them. We listed those collections in Table 1. Moreover, we extracted code smells by executing the PMD tool on each commit, as SmartSHARK does not contain the projects' code smells. PMD is an open source static analysis tool and reports the issues detected in source code.

1) SOFTWARE METRICS

Software metrics can often be used as indirect TD indicators. SmartSHARK dataset contains software metrics about the source code for each commit in Java projects. We utilized those metrics, i.e., complexity, coverage, and dependency, when evaluating their impacts on vulnerability prediction. To calculate the software metrics for each changed file in each commit, SmartSHARK uses Open-Static-Analyzer tool that creates an abstract semantic graph from the source code and applies static code analysis of projects. Open-Static-Analyzer is an open sourced version of the SourceMeter commercial tool, and a plugin for the SmartSHARK infrastructure in unification with an HPC-Cluster.

To obtain software metrics, we used code entity state and commit collections. There are, in total, 237 code metrics. The tool provides the code metrics at various levels, such as method, file, and package. When predicting code

²SmartSHARK Dataset. <https://SmartSHARK.github.io/dbreleases/>

³Apache Software Foundation. <http://apache.org>

TABLE 1. MongoDB collections of SmartSHARK dataset used in this study.

MongoDB Collection Name	Descriptions	Plugin Name
Commits	The commit information retrieved from the git log.	vcsSHARK
File Action	The changed files in each commit.	vscSHARK
Commit Changes	Changes between commits.	changeSHARK
Code Entity State	All code entity states and code metrics for each commit of a project.	coastSHARK
Refactoring	List of refactoring activities applied in the commits.	rMinerSHARK
Issue	All information added to the first issue report.	issueSHARK
Issue System	Issue system of the project.	issueSHARK
VCS System	All vcs systems from which data was collected.	vscSHARK

changes that lead to vulnerabilities, we selected 91 file-level metrics and aggregated them on commit-level to represent the commit-level code quality. There are several aggregation methods based on descriptive statistics (e.g., sum, mean, median, standard deviation, skewness), distribution parameter estimation (e.g., maximum likelihood), and econometric methods (e.g., Hoover coefficient, Gini index) [88]. Although we extracted the most common descriptive statistics, we computed the sum of each metric over all changed files in each commit.

2) REFACTORING ACTIVITIES

Refactoring activities are composed by code transformation and aim to improve the structural quality. To find commits in which refactoring activities are detected, we used commit, file actions, and refactoring collections in SmartSHARK dataset. The refactoring collection contains a list of refactoring activities that occurred in the repositories. There are 35 refactoring types that were detected via an open source tool, namely RMiner [89]. Through the use of a two-step methodology, RMiner defines various refactoring types across the history of Java projects. First, it extracts classes, methods, and fields that are added/removed/replaced between two consecutive commits based on a lightweight version of the UMLDiff algorithm [90]. Next, it extends and uses a set of rules defined by Biegel et al. [91] to discriminate the different types of refactoring activities. RMiner can detect refactoring activities with 98% and 87% of precision and recall, respectively. We aggregated refactoring activities at commit level, and there are 6,256 commits in which refactoring activities occurred.

3) BUG-INDUCING & BUG-FIXING

Bugs are defects that cause incorrect and unexpected results in software systems. To obtain bug-inducing and bug-fixing commits, we used commit, issue, and issue systems collections in SmartSHARK dataset. There is a label for each commit indicating whether it is a bug-fixing, a bug-inducing commit, or neither of the two. The issue collection lists the results of the Open SZZ execution. The Open SZZ algorithm is an implementation of the SZZ algorithm that is widely used to identify bug-inducing changes from

the history of a software project. SZZ algorithm starts by analyzing the bug-fixing commits, i.e., commits known to fix a bug reported on an issue tracking system, to trace back to their bug-introducing commits, i.e., commits containing the code changes that induce a bug into the system. We matched the commit hashes identified as bug-inducing and bug-fixing according to the issue collection with the commit hashes. There are 5,184 bug-inducing commits and 3,041 bug-fixing commits.

4) CODE SMELLS

To detect code smells, we executed PMD source code analyzer tool that has built-in rule sets and also supports the creation of custom rules. We identified four types of code smells, namely Excessive Class Length, Excessive Method Length, Excessive Parameter List, and God Class, from six projects. We matched each file with the corresponding commits in which they are affected by the code smells. In other words, code smells in every file changed in every commit were individually reported by running the PMD tool at commit level. Please note that ours is a very rigorous process, and not typical to follow, as many studies run PMD on the projects' selected snapshots or versions or patches only to find out code smells on the files due to time and memory constraints [92], [93], [94]. We used the PyDriller [95] framework to extract commit information from Git repositories instead of directly connecting to the GitHub API. PyDriller is a Python framework designed for analyzing software development processes and can be used to mine the commit history and changes in a Git repository, to measure various metrics, and to analyze code evolution. Thanks to PyDriller, we were able to save all the information from the Git log, including details about every file modified in each commit, without the need to develop a custom script for connecting to the GitHub API or parsing Git logs. Extracting code smells from every commit took approximately 108 hours using four parallel threads and resulted in a 17 MB log. We found, in total, 59,984 smelly commit-file pairs. The number of code smells ranges from 1 to 225 among the commits. We aggregated smelly commit-file pairs at commit level. Finally, we have the following information: (i) number of code smells in each

commit, (ii) detected code smell type(s) in each commit, (iii) dates when files are added or modified, and (iv) commit hash values.

C. DATA LABELING

Datasets that are created from real-world vulnerability data are generally considered to better represent industrial practice, as they reflect the complexity of real-world vulnerabilities [96]. Therefore, when building a vulnerability prediction model, we utilized real-world data for which the code and the matching vulnerability reports were obtained from real-world database and repositories. In other words, we mined projects' GitHub repositories and extracted labels of vulnerabilities that were reported by developers in NVD and ASF JIRA systems, and other project-specific websites. To this end, we performed the following three steps, summarized below.

1) EXTRACTING THE PUBLISHED VULNERABILITIES OF PROJECTS

SmartSHARK dataset does not contain projects' vulnerability data. Thus, we had to extend the dataset by curating the real-world vulnerability data analyzed from NVD for SmartSHARK projects. Vulnerabilities published in NVD are indexed according to Common Vulnerabilities and Exposures Identifier (CVE ID). Thus, we found 118 vulnerability reports as of December 2022.

2) EXTRACTING VULNERABILITY FIXING COMMITS

ASF projects, in general, do not have a specific template for security advisories, and published vulnerability reports may not contain references to CVE ID details. Hence, linking vulnerabilities to certain commits of a software project's source code was a significant and laborious challenge.

We investigated the available information for each vulnerability published in NVD and tried to find the associated fixing commit(s) in the issue tracking system or project-specific web sites. As a result of our manual search, we found 178 vulnerability fixing commits. Furthermore, we used MSR2019 dataset [97] to set the ground-truth data and validate the correctness of our mapping. MSR2019 dataset contains 624 reported vulnerabilities from 205 distinct open source Java projects and maps them onto the 1282 fixing commits.

3) EXTRACTING VULNERABILITY INDUCING COMMITS

After vulnerabilities were linked to their fixing commits, we had to find commits in which these vulnerabilities were induced in order to build a vulnerability prediction model. For this purpose, we utilized V-SZZ algorithm, which is proposed to identify vulnerability inducing changes. V-SZZ algorithm analyzes the vulnerability-fixing commits, i.e., commits in which a vulnerability reported on an issue tracking system is fixed, to trace back to their vulnerability-inducing commits, i.e., commits that include code changes inducing a vulnerability into the system. We have managed

TABLE 2. Number of extracted features based on direct/indirect TD indicators and TD mitigation methods.

TD Indicator	#Extracted Features
Software Metrics	91
Refactoring	37
Code Smells	4
Bug	2

to match vulnerability fixing commits with 190 vulnerability inducing commits.

D. DATA CLEANING

Software vulnerabilities are rarely observed compared to other software defects, even in large projects, resulting in a limited and imbalanced dataset. In our dataset, classes are imbalanced, that is, the samples labeled as vulnerable are very small compared to the entire dataset. The imbalanced data may eventually cause low prediction performance of ML models in terms of several evaluation metrics [98], [99]. To mitigate that, we applied some filtering strategies. Also, we brought the collected data into the final dataset format by applying the data cleaning steps for irrelevant code, noisy code, and duplicated code.

- We filtered out projects with a small number of real vulnerability data, as they may make the dataset more imbalanced. We selected and analysed six projects with the largest number of real vulnerability data out of 77 projects: Active-MQ, Cxf-fediz, Httpcomponents-client, Nifi, Struts, and Tika.
- We removed duplicated commits, as duplicate entries may introduce bias into a model. In other words, merged commits have two parent commits, and those commits could cause us to compute the same measure twice, thus, we removed them in our analysis.
- Meanwhile, we filtered out commits according to the number of their files when selecting. We removed commits that have more than 20 changed files due to the outlier and noise potential, which may affect the features and model training. We chose 20 by observing the number of commits and corresponding changed files, and found out that less than 5% of commits constitute more than 20 files, and an average of three files were modified in a single commit.

E. FINAL SET OF EXTRACTED FEATURES

In this subsection, we report the final set of extracted features, including software metrics, code smells, bugs, and refactoring activities, which serve as indicators of TD. Table 2 reports the number of extracted features from each set. The full list of features are available in Table 9.

1) SOFTWARE METRICS

Object oriented software metrics such as complexity, coupling, and cohesion, are closely related to TD [100],

[101] [102] and also critical in terms of several quality characteristics such as maintainability, reliability, portability, and security [30]. Thus, the software metrics can often be used as indirect TD indicators [17], [103]. We used various metrics including complexity, coverage, and dependency metrics of the projects to predict real-world vulnerabilities. These metrics are available in the SmartSHARK dataset, and we have only applied aggregation operation to have commit-level values. We refer to the websites and user manuals of the tools for detailed explanations of the calculated metrics, the documentation, libraries and API descriptions of the tools published by the researchers.

2) CODE SMELLS

Fowler defined code smells as “symptoms of poor design and implementation choices” and contributed significantly to the literature by categorizing 28 different types of smells [49], [104]. Code smells can be considered as an indicator of TD [17], [105], as they indicate areas of the codebase that may be difficult to understand, test, or maintain [48]. As shown in Table 2, we used four types of code smells that are all extracted via PMD tool at file-level. These smells are later aggregated at commit level. The features are in the form of a row vector with four dimensions each of which includes the number of specific code smell found over all files changed in a commit. *Excessive Class Length* refers that a class file is too long, containing a large number of lines of code or having too many fields or methods. *Excessive Method Length* refers that a method contains an excessive number of lines of code, making it harder to read and maintain. *Excessive Parameter List* refers that a method has too many parameters, making it harder to understand, maintain, and modify. *God Class* is a type of anti-pattern in software development that refers to a class that has become overly complex and has taken on too many responsibilities.

3) BUG-INDUCING & BUG-FIXING

Bugs are common TD indicators and can also indicate security risk [17], [106]. There are two bug-related coding activity that can be observed: (1) introduction of a bug, which could be due to human errors [107], external software dependencies [108], [109] or other factors, i.e., bug inducing, and (2) bug fixing, which may also trigger TD in software systems due to the trade-off between bug-fixing and new release pressure [110], [111]. Thus, in this study, we utilized bug-inducing and bug-fixing coding activities as two binary features of a commit.

4) REFACTORING ACTIVITIES

TD repayment methods can solve the detected causes of TD by applying various techniques such as refactoring, rewriting, and re-engineering without changing the behavior of the system, thus, mitigating or removing the principal or interest of the identified debt [55]. Meanwhile, the internal quality of the software system on which refactoring occurred

is expected to improve through mitigating or removing the accumulated TD [30], [112]. In this paper, we used 35 different types of refactoring activities as features in prediction models to analyze the impact of TD mitigation on security vulnerabilities.

F. ADDITIONAL DATASET PACKAGE

All the query scripts, collected data, and extracted additional dataset are available online [113]. Here, we also report descriptive statistics for six projects in Table 3. The projects have between 2,547 and 12,523 commits, though vulnerable commits (VC) and vulnerability-inducing commits (VIC) represent only a small portion of the total. While ActiveMQ has the most vulnerability inducing commits (73), Tika and Http Components Client have the least vulnerability inducing commits (3).

IV. METHODOLOGY

In this section, we provide our methodology that involves a comprehensive analysis of the relationship between TD and software vulnerabilities. We considered various TD indicators as features, and they were used to quantify the presence of TD in the software systems. For the purpose of predicting vulnerabilities, we employed three classifiers. We trained those classifiers using the collected data with the extracted features as input. We established a suitable training configuration to ensure optimal performance of the classifiers.

A. DIMENSIONALITY REDUCTION

The scale of the input data affects some of the models we investigated using classification algorithms. To address this issue, we applied standardization to the dataset before experimenting with the algorithms.

Performance of the algorithms may be affected when dealing with an enormous amount of features or when there is a high correlation among the features [114]. To overcome this challenge, dimensionality reduction techniques are frequently used. In this study, we used Principal Component Analysis (PCA) and Independent Component Analysis (ICA) as dimensionality reduction techniques to evaluate their impact on the performance of vulnerability classifiers.

PCA reduces the dimensionality of the original feature set by calculating the eigenvectors of the covariance matrix [115]. By linearly transforming the high-dimensional feature vector into a low-dimensional vector with uncorrelated components, the method is able to preserve most of the important information of the original feature set. On the other hand, ICA aims to obtain statistically independent components instead of transforming unrelated components [116]. In this way, ICA can potentially capture complex, nonlinear relationships between the features that PCA may not be able to capture.

In our study, we reduced the number of features to 15 when applying both PCA and ICA. This may allow us to effectively reduce the dimensionality of the feature set and simplify

TABLE 3. Descriptive statistics for six projects. (VC: Vulnerable Commit (Our labeling), VIC: Vulnerability Inducing Commit (V-SZZ labeling), VFC: Vulnerability Fixing Commit)

Project Name	Time Period	#Commits	#VC	#VIC	#VFC	#CS	#Refactoring
Tika	2007-03-31 / 2023-01-18	4,933	5	3	2	4460	4628
Nifi	2014-12-08 / 2023-01-18	5,376	35	15	6	20188	15596
Struts	2006-02-22 / 2023-01-04	6,080	249	70	21	3906	10808
ActiveMQ	2005-12-12 / 2023-01-14	12,522	277	73	17	14257	20142
HttpComponents	2005-12-21 / 2023-01-12	3,706	9	3	3	4855	12499
Cxf-Fediz	2011-12-21 / 2023-01-18	2,547	93	26	6	1292	3154
Total	—	35,164	668	190	55	48,958	66,827

the problem, while still preserving the essential information required for accurate classification.

B. CLASSIFICATION ALGORITHMS

When predicting vulnerabilities based on code smells, bug reports, refactoring activities, and software metrics, we examined the feasibility of using three classifiers, namely Support Vector Machine (SVM), eXtreme Gradient Boosting (XGBoost), and Gradient Boosted Tree (GBT). To conduct experiments, we employed Python programming language and, in particular, scikit-learn ML package, which includes the implementation of the utilized algorithms. We also applied hyperparameter tuning to each classifier to optimize their performance and presented the optimal parameters for each classifier. The employed classifiers are, in short, described below:

1) SUPPORT VECTOR MACHINE

The supervised learning model SVM aims to find the optimal hyperplane that is equidistant from both classes [117]. When the dataset can be linearly divided into classes in binary classification problems, the distinction can be made with an infinite number of hyperplanes. Therefore, SVM attempts to learn a nonlinear function by linearly mapping the data points to a high-dimensional feature space to find the linear function with the largest margin to distinguish the classes from each other. Thus, the misclassification error is reduced when new data is encountered. We defined the optimal parameter set of SVM by k-fold cross validation and performed the classifier with poly kernel as it outperformed other kernels to predict vulnerabilities [117].

2) GRADIENT BOOSTED TREE

GBT can be applied for classification or regression problems, based on the principle that many weak learners can together form a more accurate estimate. When predicting target labels, it utilizes an ensemble of decision trees to optimize the prediction result of a model with consecutive steps in the learning phase. The algorithm incrementally adds new models with iterations until no improvement is detected and transforms weak learners into strong learners. Each iteration of the trees covers tuning the values such as the coefficient,

weight, and bias applied to each input variable for minimizing the difference between the predicted and actual, namely loss function. Throughout the experiments, we performed hyperparameter tuning on the GBT using grid search to prevent overfitting, as lower variance would lead to more accomplished results in samples outside of the training data.

3) EXTREME GRADIENT BOOSTING

XGBoost is a tree based ensemble ML method designed to increase model performance and computational speed [118]. It uses multiple decision trees to predict an outcome based on central boosting techniques through system optimization such as parallel processing, tree punning, handling missing values, and regularization overfitting/bias. XGBoost has many hyperparameters, therefore, we implemented the grid search with a limited combination of parameters to achieve optimal performance [118].

C. TRAINING CONFIGURATION

1) SAMPLING METHODS

Many traditional ML classifiers are less powerful as a result of the skewed distribution, when trying to predict minority class samples in an imbalanced dataset [119]. As detailed in Section III-D, we applied some filtering strategies. However, the ratio of vulnerable samples to the entire dataset is still 0.01%. Thus, we applied additional steps when labeling the data, that is, we selected commits that have files labelled as vulnerable at least once in their entire change histories. For instance, as seen in Figure 2, we found that commit 28067a2 is the vulnerability-fixing commit, and the vulnerability was induced at commit ee9bd94. So this vulnerability lives in the system from the moment it is induced until it is fixed. That's why we labeled all the commits after which the affected files in the inducing commit were changed as vulnerable. So, we labelled commits 44c9ea0, 710171f, and f60585a as vulnerable in which *splitJson*, *TestsplitJson*, *TestsplitXml*, and *SplitXml* files are modified. With this operation, class distributions were more balanced.

As the final dataset is relatively balanced compared to a real-life scenario, the prediction performance that we would observe at the end of this study would not be generalizable to our contexts. To evaluate the robustness and generalizability

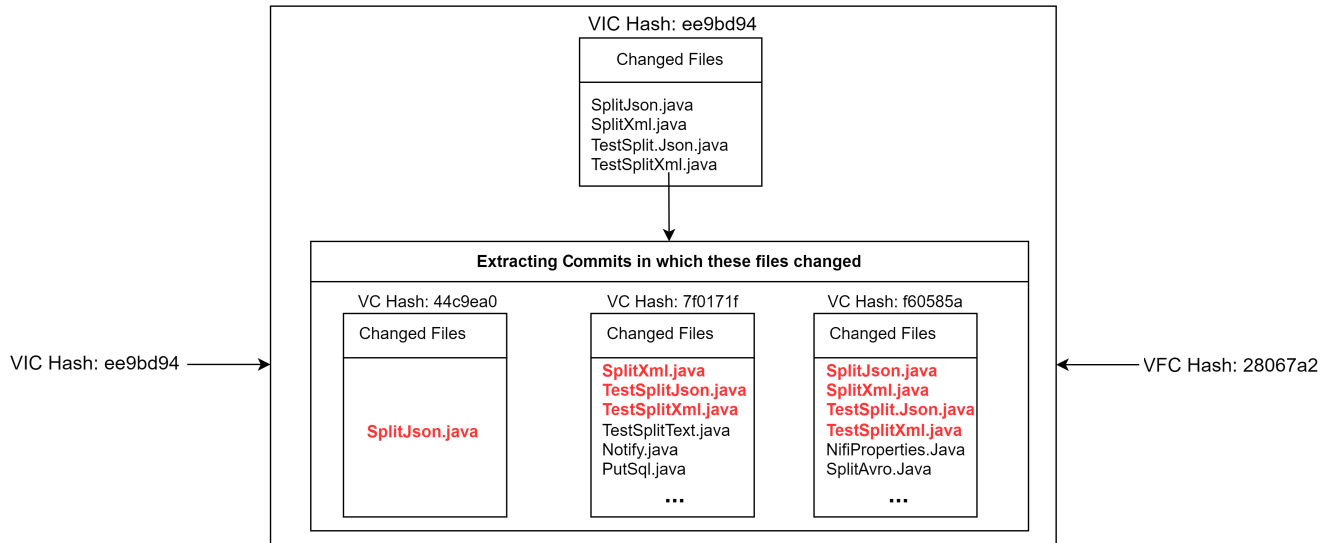


FIGURE 2. Labelling Vulnerable Commits. (VC: Vulnerable Commit, VIC: Vulnerability Inducing Commit, VFC: Vulnerability Fixing Commit).

of our models, we have performed additional experiments by deliberately unbalancing the ratio of the test set. In other words, we added commits that do not contain any vulnerable files throughout their history into the test set such that vulnerable and non-vulnerable commit ratios become 30%-70%, 20%-80%, 10%-90% respectively.

2) TRAIN-TEST SPLIT

Our dataset, prepared for supervised learning, consists of 1,321 samples belonging to six real-world projects in total. We randomly split 80% of the data for training and 20% for testing. Random splitting of the dataset can result in extremely different ratios in the target variable. In particular, the distribution of target variables can potentially affect the quality of model predictions as our dataset is imbalanced and the number of samples in vulnerable class is quite small. To overcome this challenge, we ensure that the data is split with stratification. To achieve more robust results, we repeated the experiments 10 times each using a different random seed on the order of the samples, and we computed the average of the overall performance scores.

3) HYPER-PARAMETER TUNING

To improve the prediction performance of models, a hyper-parameter tuning procedure must be carried out before learning. An external property of the model known as a model hyper-parameter cannot be inferred from data during training. We used GridSearchCV, a Python version of the Grid-search method, to fine-tune our models as much as feasible [120]. Grid-search is frequently used to execute an exhaustive search over predetermined parameter values for an estimator in order to identify the model hyper-parameters that produce the most accurate predictions. To assess a parameter setting, we used the F1 score as the estimator's objective function.

TABLE 4. Confusion matrix for two-class classification.

Actual/Predicted	Vulnerable	Non-vulnerable
Vulnerable	<i>True Positive (TP)</i>	<i>False Negative (FN)</i>
Non-vulnerable	<i>False Positive (FP)</i>	<i>True Negative (TN)</i>

We performed hyper-parameter selection on every classifier under investigation during training.

D. EVALUATION METRICS

We evaluated and compared the predictive performance of the analyzed classifiers in terms of Precision, Recall and F1-score. We computed the measures via the confusion matrix.

Precision (shown in Eq. 1), is the percentage of actual vulnerability-inducing commits among the predicted vulnerability-inducing commits. It is important when FPs are costly and should be decreased.

$$Precision(P) = \frac{TP}{TP + FP} \quad (1)$$

Recall (shown in Eq. 2) provides the percentage of accurately predicted vulnerability-inducing commits. It is important when FNs are critical.

$$Recall(R) = \frac{TP}{TP + FN} \quad (2)$$

False Positive Rate (shown in Eq. 3) measures the percentage of non-vulnerable commits incorrectly predicted as vulnerable. It is critical when minimizing false positives is a priority.

$$FalsePositiveRate(FPR) = \frac{FP}{FP + TN} \quad (3)$$

TABLE 5. Software vulnerability prediction performance using different TD indicators and classification models.

	SM + CS + Bug + Ref			SM + CS			SM + Bug			SM + Ref			SM		
Model Name	R(%)	P(%)	F1(%)	R(%)	P(%)	F1(%)	R(%)	P(%)	F1(%)	R(%)	P(%)	F1(%)	R(%)	P(%)	F1(%)
XGB	98.7	95.9	97.3	97.4	96.9	97.1	79.8	76.5	78.1	75	67.2	70.8	79.4	74	76.6
GBT	98.7	93.9	96.2	95.5	91.7	93.6	82.5	75	78.6	77.1	69	72.7	80.1	73.9	76.9
SVM	99	70.7	82.5	98.1	62	76	93.8	58.5	72.1	93.9	60.2	73.3	98	57.1	72.1
XGB+PCA	89.4	81.5	85.2	83.3	78.9	81	73.9	64.1	68.6	71.1	66.4	68.6	72.1	66.3	69.1
GBT+PCA	91.9	83.1	87.2	84.9	76.8	80.6	76.5	64.8	70.1	69.6	65.6	67.5	74.1	66	69.8
SVM+PCA	99.3	64.4	78.2	99.1	60.4	75	95.7	56.9	71.3	98.6	59.6	74.2	98.8	56.5	71.9
XGB+ICA	88.9	79.8	84.1	82.4	77.1	79.6	74.1	65.5	69.4	72.9	64.8	68.5	70.8	65.3	67.9
GBT+ICA	90.8	80.8	85.5	85.3	75.2	79.9	74.9	64.3	69.2	69.3	63.7	66.2	72.8	65.7	69
SVM+ICA	98.8	65.4	78.7	98.9	60.7	75.3	95.1	57	71.3	96.6	59.6	73.7	98	56.7	71.9

F1-Score (shown in Eq. 4) measures the trade-off between recall and precision.

$$F1 - Score(F1) = \frac{2 * recall * precision}{recall + precision} \quad (4)$$

Also, we employed Scott-Knott Effect Size Difference (ESD) test to group both feature sets and ML models based on their effect size differences, which provides a nuanced perspective on their relative performance.

V. ANALYSIS OF THE RESULTS

In this study, when answering RQs, we evaluated the performance of 9 ML models using direct/indirect TD indicators as features to predict vulnerable code changes. To this end, we used SVM, GBT, and XGB classifiers. We also performed PCA and ICA separately for each model. When measuring the performance of the models, we used the recall, precision, false positive rate, and F1-Score. We report our experimental results for all trained vulnerability models in Table 5. Our research methodology involves a comprehensive evaluation of the performance of the vulnerability prediction model using both balanced and unbalanced test sets created with various rates (70%,80%,90%). The results for each test set created are shown in separate tables. This can provide valuable insights into the robustness and generalizability of the model for highly imbalanced datasets.

According to our results, direct/indirect TD indicators within software development, such as software metrics, code smells, bugs, or refactoring activities, contribute to the occurrence of vulnerabilities in software projects. Moreover, we observe that combining software metrics with code smells achieves higher success rates in predicting vulnerabilities than refactoring and bugs. In the subsections, we discuss the answers to the research questions in detail and address the impact and effectiveness of our proposed methods.

A. RQ1: HOW CAN WE PREDICT SOFTWARE VULNERABILITIES WITH RESPECT TO CODING ARTIFACTS LEADING TO TD?

Here, we compare the performance of different feature set combinations, including Software Metrics (SM), Code Smells (CS), Bugs (Bug), and Refactoring (Ref) features. In addition, we evaluate the distinct contribution of code smells to the prediction of vulnerable commits. As seen in

Table 5, using only SM provides good results in terms of recall, precision, and F1-Score with rates of 98.9%, 74%, and 76.9%, respectively. Those rates indicate that software metrics alone can be a valuable indicator of vulnerabilities. Combining software metrics with code smells (SM+CS) significantly improves the results by achieving 99.1% recall rate, the 97.1% F1 score rate, and the highest precision rate with 96.9%. Nevertheless, SM+CS+Bug+Ref feature set combination achieves slightly higher recall (99.3%) and F1-Score (97.3%) rates compared to SM+CS. Combinations of all features are effective in identifying a high proportion of all software vulnerabilities.

We further analyze the feature sets with the Scott-Knott ESD test by ranking their precision, recall, and F1-Score rates, clustering them in non-overlapping groups, and identifying significant differences. Figure 4 shows the results of the Scott-Knott test among all feature set combinations. Different colors (blue, green, purple, and orange) of the plot indicate different Scott-Knott test ranks, and the coloring of the violins represents distinct clusters of the feature set combinations. When two feature sets are not significantly different, they are illustrated with the same color. In Figure 4b, SM+CS+Bug+Ref combination (shown in blue) consistently achieves high precision scores, while SM+CS (shown in orange) also exhibits promising performance. Although we see different precision ratios in Table 5 for SM, SM+Bug, and SM+Ref combinations, they are grouped in the same cluster according to the Scott-Knott results. Similarly, according to Scott-Knott results, SM+CS+Bug+Ref feature combination and SM+CS again have high recall rates, and are significantly different from other models, while there is no significant difference between the recall results of other models (SM, SM+Bug, and SM+Ref). Furthermore, SM+CS+Bug+Ref combination (shown in green) consistently achieves high F1 scores, indicating its effectiveness in predicting software vulnerabilities. In addition, SM+CS (shown in orange) falls into a different ESD group with SM+CS+Bug+Ref.

More importantly, feature sets that do not contain code smells (i.e., SM, SM+Bug, SM+Ref) provide good prediction results in terms of recall, precision, and F1-Score. However, it is clearly seen that when code smells are added (i.e., SM+CS, SM+CS+Bug+Ref), they significantly contribute to the performance of vulnerability prediction.

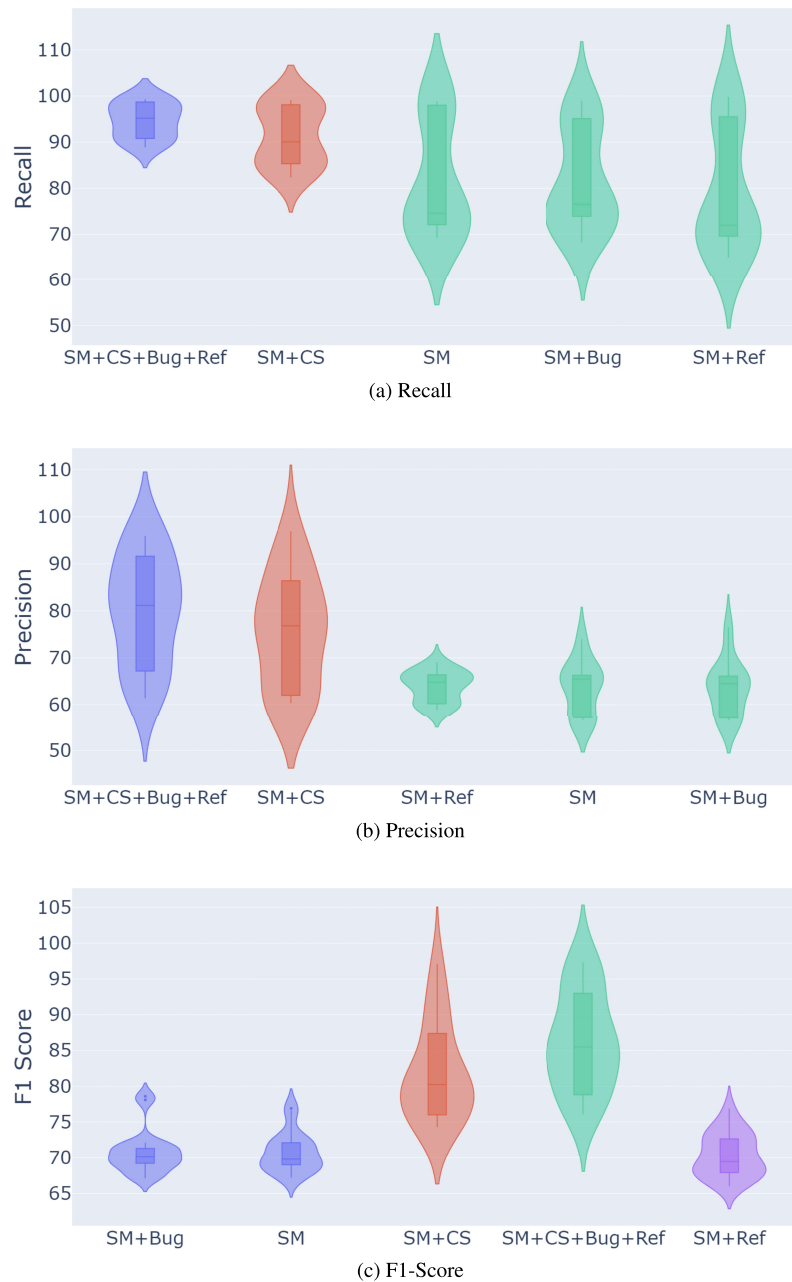


FIGURE 3. Scott-Knott test results for feature set combinations.

So, we confirm the findings of previous studies reporting a strong relationship between code smells and software vulnerabilities. In [14] and [15], authors identified a statistically significant correlation between code smells and software vulnerabilities. Similarly, our results also demonstrate that code smells are strong indicators of security risks, as the inclusion of code smells enhances the performance of vulnerability prediction models. Furthermore, our study aligns with the findings of [17] and [19], which explored the relationship between TD indicators and software security, by providing empirical evidence of a strong correlation between TD and vulnerabilities.

RQ1: Analysis of feature combinations shows that the choice of a feature set has a significant impact on the performance of ML models to predict vulnerable commits. SM+CS+Bug+Ref and SM+CS feature sets appear to be the most effective indicators of vulnerabilities. In particular, incorporating code smells (CS) into software metrics (SM) strongly influences the model performance.

1) EFFECT OF MACHINE LEARNING ALGORITHMS

Data-centric and model-centric AI represent two distinct approaches in the development of AI systems [121].

Data-centric approaches focus primarily on the quality, consistency, and pre-processing of data. In this approach, the emphasis is on refining the dataset to improve model performance, with the understanding that better data leads to more accurate and reliable outcomes. This could involve techniques like data augmentation, cleaning, and ensuring diversity and representativeness in the dataset. On the other hand, model-centric approaches prioritize the algorithmic aspect, with a focus on selecting and tuning the right machine learning algorithms to optimize performance. In this paradigm, the effort is more on improving the model architecture, tweaking hyperparameters, and employing sophisticated algorithms to achieve the desired results. The choice between a data-centric or model-centric approach often depends on the specific problem, available resources, and desired outcomes.

We have adopted a *data-centric approach*, emphasizing the meticulous analysis of our data set and the labeling of the data, rather than focusing on finding the best model architecture [96], [122]. Having said that, we picked three ML classifiers, namely SVM, GBT and XGB, and evaluated our analysis on these classifiers' performance. While comparing the performances of ML models, we again applied the Scott-Knott test to support our results. According to the results in Table 5, SVM+PCA outperforms all other ML models by achieving the highest score in terms of recall with 99.3% but its precision (64.4%) and F1-Score (78.2%) are lower. On the other hand, we report the best results in terms of precision and F1-Score with XGB and GBT classifiers across all feature sets.

The results show that applying dimensionality reduction techniques to some models improves overall performance. However, the improvements are not consistently significant across all models and all feature sets. In Table 5, we observe that when PCA is applied with XGB and GBT classifiers, the performance metrics often show a decrease in recall, precision, and F1-score compared to the non-PCA models. This reduction in performance may be due to the linear nature of PCA, which may not effectively capture underlying patterns in data when dealing with complex, nonlinear relationships. On the other hand, while PCA is applied with SVM, it improves the recall metric, while it cannot improve the precision and F1-scores. Meanwhile, models with ICA also show a decrease in performance compared to models without ICA.

2) EFFECT OF USING MORE REALISTIC TEST SET

As stated in Section IV-C, the test set must align with real-life scenarios for the results to have practical value. Therefore, we have created an imbalanced test data and evaluated the performance of the models with this version. In this section, we will discuss the ratio of 90%-10% for training-test set, while results for other ratios are provided in Table 10 and Table 11 in Appendix, where similar conclusions are obtained. When the ratio is 90%-10%, as shown in Table 6, XGB model achieves the highest recall,

precision, accuracy, and F1-score when all the features (SM, CS, Bug, Refactoring) are used.

In general, precision in all models decreases, as expected, leading to a drop in the F1 score. This indicates that the false positive rate (FPR) increases. In our previous studies [26], [27], we did not apply additional steps when labeling the training data, resulting in an imbalanced dataset. Although we applied traditional under-sampling and over-sampling on the imbalanced datasets, we could not reach a precision rate of above 4%(in [26]) and 11.5%(in [27]). In this study, we observe that our labeling approach, as detailed in Section IV-C, is highly effective. As shown in Table 6, for the 90%-10% ratio, we have achieved precision rates of 45.2

On the other hand, when examining the results in terms of feature sets, we have achieved the best precision rate with the combined and SM+CS sets. This result emphasizes that CS remains the most effective feature set for predicting vulnerable commits in real-world scenarios. Regarding ML algorithms, XGB stands out as the most successful model and the most robust against imbalanced data.

B. RQ2: HOW DOES V-SZZ+ IMPROVE VULNERABLE COMMIT PREDICTION COMPARED TO V-SZZ?

When answering RQ2, we evaluate the performance of software vulnerability prediction models across three configurations:

- V-SZZ+/V-SZZ+: Both training and testing use V-SZZ+ labeled data.
- V-SZZ+/V-SZZ: Training uses V-SZZ+ labeled data, but testing uses V-SZZ labeled data.
- V-SZZ/V-SZZ: Both training and testing use V-SZZ labeled data.

We assessed the performance in terms of recall, False Positive Rate (FPR), and F1-score. The results are presented using FPR instead of precision because V-SZZ considers newly labeled instances as safe, which can lead to a high number of false negatives (vulnerable instances misclassified as safe). While FPR is moderately higher in the V-SZZ+ setting, this trade-off is acceptable since minimizing missed vulnerabilities (high recall) is often prioritized over reducing false alarms. In contrast, the traditional V-SZZ labeling approach yields very low recall and F1 scores, despite a low FPR, indicating that it fails to capture the majority of vulnerable commits due to its snapshot-based and more limited labeling scope.

V-SZZ+ approach, which labels vulnerability-inducing commits and all subsequent commits containing vulnerabilities until they are fixed, significantly improves vulnerable commit prediction compared to the standard V-SZZ approach. The V-SZZ+/V-SZZ+ configuration achieves the highest recall (up to 98.80%) and F1-scores (up to 76.90%). It demonstrates its ability to effectively identify vulnerable commits with a strong balance of precision and recall.

Moreover, V-SZZ+/V-SZZ configuration, despite benefiting from V-SZZ+ during training, suffers from a high

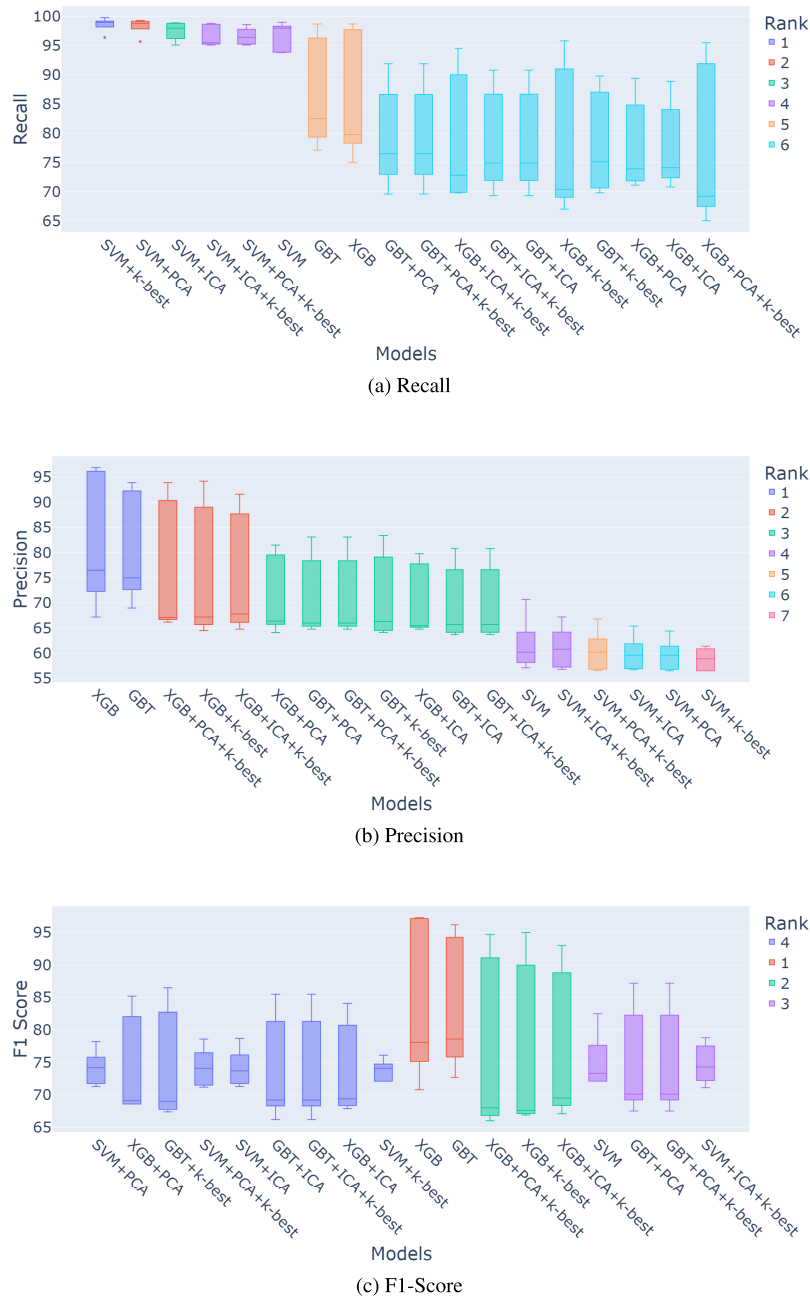


FIGURE 4. Scott-Knott test results for ML models.

FPR (e.g., 57.54%–78.82%) and lower F1-scores (e.g., 28.41%–32.47%) due to the inconsistency of testing with V-SZZ labels. This mismatch highlights the importance of using V-SZZ+ consistently for both training and testing to ensure accurate predictions. The V-SZZ/V-SZZ configuration performs the worst, with extremely low recall (e.g., 1.58%) and F1-scores (e.g., 3.04%). This result indicates that standard V-SZZ fails to capture the persistence of vulnerabilities across commits and limits its utility for vulnerability prediction.

In conclusion, V-SZZ+ significantly enhances vulnerable commit prediction compared to standard V-SZZ by providing a more comprehensive and accurate labeling of vulnerable commits. Its ability to achieve high recall and F1-scores makes it a superior approach for identifying vulnerabilities in real-world software projects and supports improved software security and maintenance practices. The exact impact of FPR in V-SZZ+/V-SZZ+ would further clarify its precision, but the current results strongly favor V-SZZ+ as a robust and effective method for vulnerability prediction.

TABLE 6. Software vulnerability prediction performance when the test set class ratio is imbalanced(10%-90%).

	SM+CS+Bug+Ref			SM+CS			SM+Bug			SM+Ref			SM		
Model Name	R	P	F1	R	P	F1	R	P	F1	R	P	F1	R	P	F1
XGB	99	42.7	59.6	96.3	45.2	61.5	81.8	31.5	45.4	74.7	25.9	38.5	79.6	31	44.5
GBT	98.6	39.5	56.4	96	41.7	58.2	83.1	30.7	44.8	77.4	27.1	40.1	83.3	30.3	44.5
SVM	99.2	24.6	39.4	97.9	21	34.6	93.1	21.3	34.6	93.7	20.1	33.1	97.8	20.4	33.7
XGB+PCA	88.6	31.9	46.9	82.3	31.4	45.4	74	24	36.3	73.2	24.5	36.6	71.6	24.5	36.5
GBT+PCA	92.8	32.7	48.3	84.4	30.5	44.7	77.1	24.5	37.2	73.7	24.8	37	74	25.2	37.6
SVM+PCA	99.6	21.7	35.7	98.9	20.5	33.9	96.9	20.3	33.6	98.4	20.1	33.4	98.4	20.2	33.5
XGB+ICA	88.2	30.8	45.6	84.1	30.8	45	75.3	25.2	37.8	79.5	24.4	37.3	69.6	24.6	36.3
GBT+ICA	90.6	31.6	46.9	84.9	30.1	44.4	76.7	24.5	37.2	74.2	24.1	36.3	70.2	25.2	37.1
SVM+ICA	99	22.1	36.1	98.6	20.6	34.1	95.3	20.2	33.3	97.4	20.2	33.4	97.3	20.2	33.4

TABLE 7. Predicting vulnerable commits with different train/test configurations.

	V-SZZ+/V-SZZ+			V-SZZ+/V-SZZ			V-SZZ/V-SZZ		
Model Name	R(%)	FPR(%)	F1(%)	R(%)	FPR(%)	F1(%)	R(%)	FPR(%)	F1(%)
XGB	79.40	32.86	76.60	72.89	57.54	31.99	11.05	3.37	17.31
GBT	80.10	35.61	76.90	74.47	57.81	32.47	7.63	4.17	11.77
SVM	98.00	94.80	72.10	95.26	76.58	28.41	5.00	1.93	8.69
XGB+PCA	72.10	49.49	69.10	66.32	59.89	28.71	4.47	1.50	7.98
GBT+PCA	74.10	48.98	69.80	69.47	61.39	29.44	8.68	2.78	13.53
SVM+PCA	98.80	98.47	71.90	97.63	78.82	28.55	1.58	0.32	3.04
XGB+ICA	70.80	47.86	67.90	73.42	58.02	31.97	3.16	1.55	5.57
GBT+ICA	72.80	49.59	69.00	72.63	58.89	31.40	10.00	3.26	15.67
SVM+ICA	98.00	96.43	71.90	97.37	77.71	28.72	1.84	0.48	3.47

RQ2: V-SZZ+ achieves higher recall and F1-scores by labeling vulnerability-inducing and subsequent commits, and significantly outperforms standard V-SZZ in predicting vulnerable commits.

C. RQ3: HOW DOES DETECTING CODE SMELLS AT EVERY COMMIT PERFORM IN IDENTIFYING VULNERABLE COMMITS?

To address RQ3, we analyze the results presented in Table 8, which compares the performance of the proposed commit-level code smell detection approach (referred to as “Our approach”) against the traditional snapshot-based approach for software vulnerability prediction. The performance metrics used to evaluate the models are recall, precision, and F1-score across various ML models, including XGB, GBT, SVM, and their variants with PCA and ICA. The results demonstrate that our approach, which involves fine-grained, commit-level code smell detection, consistently outperforms the snapshot-based approach across all models and metrics. For instance, the XGB model under our approach achieves recall of 97.4%, precision of 96.9%, and F1-score of 97.1%, compared to 84.36%, 79.49%, and 81.80%, respectively, for the snapshot-based approach. This represents an improvement of approximately 13.04% in recall, 17.41% in precision, and 15.30% in F1-score. Similarly, the GBT model shows substantial gains, with our approach yielding a recall of 95.5%, precision of 91.7%,

and F1-score of 93.6%, compared to 86.93%, 82.01%, and 84.33% for the snapshot-based approach.

The performance of our approach can be attributed to its ability to capture the temporal evolution of code smells and their relationship with vulnerability-inducing changes at a granular level. By analyzing every file modified in each commit, our approach provides a detailed mapping of smelly commit-file pairs, enabling a more precise identification of vulnerable commits. This contrasts with snapshot-based approaches, which typically analyze the codebase at specific points in time, often missing intermediate states that may introduce or exacerbate vulnerabilities.

The fine-grained nature of our approach provides a more comprehensive perspective on the relationship between TD and software vulnerabilities. By tracking code smells at the commit level, we can better correlate specific changes with the introduction of vulnerabilities, offering insights into how TD accumulates over time. For example, the higher recall and F1-scores in our approach suggest that it is more adept at identifying commits where code smells (e.g., complex methods, god classes) align with vulnerability-inducing changes. Also, this temporal alignment is critical for understanding the causal links between TD and vulnerabilities, which snapshot-based approaches may overlook due to their static nature. The results have significant implications for software development practices. Our approach’s ability to detect vulnerabilities with higher accuracy at the commit

TABLE 8. Combined software vulnerability prediction performance: Snapshot based versus Our approach.

Model Name	Snapshot based approach			Our approach		
	R (%)	P (%)	F1 (%)	R (%)	P (%)	F1 (%)
XGB	84.36	79.49	81.80	97.4	96.9	97.1
GBT	86.93	82.01	84.33	95.5	91.7	93.6
SVM	97.23	60.03	74.22	98.1	62.0	76.0
XGB+PCA	76.73	68.59	72.40	83.3	78.9	81.0
GBT+PCA	77.82	69.10	73.13	84.9	76.8	80.6
SVM+PCA	98.32	58.62	73.45	99.1	60.4	75.0
XGB+ICA	77.13	68.10	72.26	82.4	77.1	79.6
GBT+ICA	78.12	69.81	73.69	85.3	75.2	79.9
SVM+ICA	97.43	59.03	73.52	98.9	60.7	75.3

level enables developers to address issues proactively during the development process, rather than relying on periodic codebase snapshots.

RQ3: Commit-level code smell extraction outperforms traditional snapshot-based approaches in identifying vulnerable commits. This fine-grained approach effectively captures temporal changes in code smells and enhances the performance of vulnerability prediction.

VI. THREATS TO VALIDITY

In this section, we discuss possible threats to the validity of our study, including threats to external, construct, internal, and conclusion validity.

A. INTERNAL VALIDITY

Internal validity refers to the degree that a study provides credible and accurate evidence for establishing a causal relationship between variables. In this study, it concerns the ability to establish a causal relationship between TD and software vulnerabilities accurately. Possible threats to internal validity that may have impacted our findings are primarily associated with the measurement of constructs at various levels of granularity or the comparison of different items with one another. We conduct commit-level analysis in terms of both TD and vulnerabilities to avoid that problem. Furthermore, potential threats to internal validity include confounding variables that may influence the relationship, such as other factors impacting software security or unobserved variables not accounted for in the analysis. To mitigate this threat, we employed a rigorous research design and performed a comprehensive analysis using direct and indirect TD indicators. Moreover, in a software project, we examined software metrics, code smells, bugs, and refactoring activities that can affect the relationship between TD and vulnerabilities. It is also possible that the TD and its mitigation method in a file are affected by the decisions and experience of the developers. But, it is beyond

the scope of this post to investigate the causality between developers and TD.

B. EXTERNAL VALIDITY

External validity refers to the generalizability of the findings beyond the specific context of the studied projects. Potential threats to external validity relate to the extent to which findings can be applied to different environments, such as various software projects or programming languages. However, this study solely focuses on open source Java projects belonging to ASF. Therefore, we cannot establish the generalizability of our study outcomes to industrial projects, or to projects developed in different programming languages or ecosystems. It is essential to consider the diversity of projects, development practices, and organizational contexts to enhance the external validity of the study. Please note that the data set employed in our investigation provides a comprehensive source of information, containing 47,303 pull requests, 163,057 issues, 2,987,591 emails, and 366,322 commits. We also chose six projects with the greatest quantity of real vulnerability data. As a result, we can confidently claim that our RQs have been addressed on the most extensive and diverse data set available in this field. Moreover, to mitigate threats related to reliability, we took several measures. In Section III, we provide a comprehensive description of the data preparation pipeline, ensuring transparency and reproducibility and in Section IV, we thoroughly outline the analysis steps and methods employed in the study. Additionally, as we use a publicly available dataset, provide all the query scripts, collected data and extracted additional dataset in detail, other studies can also replicate or refute our findings on the relationships between TD and software vulnerabilities. To enhance the reliability and balance of our dataset, we selected commits that have files marked as vulnerable at least once throughout their change histories as explained in detail in Section IV-C. Furthermore, in order to assess the robustness and generalizability of our models, we conducted supplementary experiments where we

TABLE 9. Extracted features based on direct/indirect TD indicators and TD mitigation methods.

TD Indicator	Extracted Feature
Software Metrics	WMC, NL, NLS, MigratingToJUnit4 Rules, TNG, Migration Rules, TNLPA, CI, LLDC, NLPM, Type Resolution Rules, PDA, NPM, NLA, AD, Jakarta Commons Logging Rules, TCD, NS, Migration13 Rules, RFC, Runtime Rules CCO, NLG, NOP, String and StringBuffer Rules, TLOC, Comment Rules, NA, Brace Rules, LCOM5, CD, NOA, Android Rules, TCLOC, TNLM, Code Size Rules, Strict Exception Rules, CLC, CLOC, LOC, TNM, NM,
	Java Logging Rules, Naming Rules, TNLPM, LLOC, TNA, Coupling Rules, LDC, NOI, NOC, Migration15 Rules, CC, TNLS, NLPA, PUA, JavaBean Rules, Security Code Guideline Rules, DIT, NLM, CCL, Import Statement Rules, Unnecessary and Unused Code Rules, TNPA, NPA, NOD, Finalizer Rules, CBO, NII, TNOS, Clone Implementation Rules, Design Rules, TNPM, Migration14 Rules, NG, J2EE Rules, CLLC, Basic Rules, Optimization Rules, TLLOC, TNLA, Empty Code Rules, NOS, CBOI, JUnit Rules, NLE, TNS, DLOC, TNLG, Controversial Rules
Refactoring	Change Return Type, Change Variable Type, Extract and Move Method, Extract Attribute, Extract Class, Extract Interface, Extract Method, Extract Subclass, Extract Superclass, Extract Variable, Inline Method, Inline Variable, Merge Attribute, Merge Parameter, Merge Variable, Move and Rename Attribute, Move and Rename Class, Move Attribute, Move Class, Move Method, Parametrize Variable, Pull Up Attribute, Pull Up Method, Push Down Attribute, Push Down Method, Rename Attribute, Rename Class, Rename Method, Rename Parameter, Rename Variable, Replace Attribute, Replace Variable with Attribute, Split Attribute, Split Parameter, Split Variable
Code Smells	GodClass, ExcessiveClassLength, ExcessiveParameterList, ExcessiveMethodLength
Bug	Bug Inducing, Bug Fixing

intentionally created an imbalanced ratio within the test set. This involved varying the rates at 70%, 80%, and 90%. By doing so, we aimed to simulate real-world scenarios where the data distribution may significantly differ from the training data.

C. CONSTRUCT VALIDITY

Threats in construct validity concerns the accurate measurement and operationalization of TD indicators and software vulnerabilities. It is crucial to ensure that the chosen indicators effectively capture the concepts of interest and provide reliable and valid measurements. There may be alternative definitions or measures of TD and vulnerabilities that were not considered in the study, potentially impacting the construct validity. The dataset includes software metrics collected by SourceMeter tool, and hence metrics that the tool could not identify are excluded. Nevertheless, SourceMeter is able to extract the majority of the most popular metrics. Also, the dataset utilized in this study includes refactoring actions captured by the RMiner tool. However, it is important to note that the tool may not identify certain refactoring types, as it primarily focuses on the most commonly employed ones by developers. Additionally, we recognize the significance of accurately calculating precision and recall to ensure precise and comprehensive detection of refactorings. In this regard, the RMiner tool exhibits a precision of approximately 98% and a recall of 87%. Furthermore, RMiner is considered one

of the most advanced and widely adopted tools for detecting refactoring activities in the field.

D. CONCLUSION VALIDITY

Conclusion validity relates to the reliability of the inferences and conclusions drawn from the analysis. Threats to conclusion validity may arise from statistical issues, such as small sample sizes or low statistical power, which can limit the generalizability of the findings. Care should be taken to appropriately analyze and interpret the results to minimize the risk of drawing inaccurate or overly broad conclusions. Our conclusions are limited to 6 projects implemented Java, but the prediction results are statistically compared among several representations and models, and validated through non-parametric significance tests.

VII. CONCLUSION

TD and software security may have a strong relationship in the sense that both can affect the quality attributes of a software system including security maintainability, sustainability and evolvability. TD refers to extra maintenance cost and improvement of software projects due to past decisions that can make it harder to add new features or fix bugs. Software security, on the other hand, refers to the measures for protecting against security risks by building secure software. In this context, software systems with high TD may be more vulnerable to attacks as it is more difficult to understand, test, update, and maintain. In this study,

TABLE 10. Software Vulnerability Prediction results of different code representation approaches and classification models. Test set created with rate of 70%.

Model Name	SM + CS + Bug + Ref.				SM + CS				SM + Bug				SM + Ref.				SM			
	R	P	Acc	F1	R	P	Acc	F1	R	P	Acc	F1	R	P	Acc	F1	R	P	Acc	F1
XGB	98.6	72.4	84.2	83.5	97.3	76.3	86.8	85.5	79.3	57.5	68.3	66.6	73.7	47	56.2	57.3	81.4	58.2	69	67.8
GBT	98.5	70	82.2	81.8	95.8	70.7	82.4	81.3	81.6	56.1	67.1	66.4	78.7	50.3	60.3	61.2	82.3	56.4	67.4	66.9
SVM	98.8	47.8	56.1	64.5	98.4	41.9	44.8	58.8	93.7	41.9	45.6	57.9	95	40.9	43.2	57.2	97.8	40.6	41.8	57.3
XGB+PCA	89.3	57.5	69.1	69.9	83.3	57.5	68.7	68.1	74.7	47.2	56.5	57.8	71.6	47	56.1	56.6	72.8	47.9	57.3	57.7
GBT+PCA	91.4	58.6	70.4	71.4	84.9	55.5	66.7	67.1	77.4	47.5	56.7	58.8	70	45.5	54.2	55	75.3	48.6	58.4	59.1
SVM+PCA	99.2	42.6	459	59.6	99.3	40.9	42.2	57.9	96.1	40.3	41.5	56.8	98.7	40.3	41.1	57.3	98.6	40.1	40.6	57
XGB+ICA	89.5	57.1	68.7	69.7	82.5	56.6	67.5	67.1	75.6	48.1	57.5	58.7	73.9	45.6	54.3	56.3	74.1	48.7	58.5	58.7
GBT+ICA	91.7	57.8	69.6	70.9	85.4	55	66.1	66.9	76.7	47.2	56.4	58.4	68.9	43.7	52	53.5	74.3	49.8	59.8	59.6
SVM+ICA	98.8	43.4	47.6	60.3	99.1	41.2	42.9	58.2	95.6	40.4	41.8	56.8	974	40.4	41.5	57.1	98.1	40.3	41.1	57.1

TABLE 11. Software Vulnerability Prediction results of different code representation approaches and classification models. Test set created with rate of 80%.

Model Name	SM + CS + Bug + Ref.				SM + CS				SM + Bug				SM + Ref.				SM			
	R	P	Acc	F1	R	P	Acc	F1	R	P	Acc	F1	R	P	Acc	F1	R	P	Acc	F1
XGB	99.1	58.4	78.3	73.4	97.8	62.6	81.7	76.3	80.1	45.8	65.3	58.2	78.3	39.6	56.9	52.5	80	46.5	66	58.7
GBT	98.6	55.1	75.2	70.6	95.6	57.2	77	71.5	83.8	44.9	64	58.5	76.9	39.4	56.8	52	81.3	45.2	64.5	58
SVM	99.3	36.2	47.1	531	97.8	31.5	35.3	47.7	93.8	32.1	38.2	47.8	93.1	30.7	33.9	46.2	98.2	30.8	32.7	46.9
XGB+PCA	88.1	44.3	63	58.9	83	45.5	64.8	58.7	73.5	35.4	51.5	47.7	73.4	37.1	53.6	49.2	70.9	36.5	53.9	48.1
GBT+PCA	92.2	45.8	64.7	61.2	84.4	44.2	63.1	58	76.2	36	51.9	48.9	72.4	36.7	53.2	48.6	73.8	37.5	55	497
SVM+PCA	99.3	32.2	36.8	48.6	99	30.7	32.3	46.9	95.6	30.5	32.9	46.3	98.3	30.7	31.8	46.8	99	30.4	31.3	46.5
XGB+ICA	87.8	44.4	63	58.9	82.7	44.2	63.1	57.6	76.3	37.1	53.8	49.9	72.4	35	50.7	47.1	69.7	36.4	53.9	47.8
GBT+ICA	88.5	44.3	62.9	59	84.8	43.1	61.6	57.2	77.2	36.1	51.6	49.1	67.9	34.5	50.8	45.7	72.2	37.7	55.6	49.5
SVM+ICA	98.6	32.8	38.6	49.2	98.7	30.9	33	47.1	94.9	30.6	33.3	46.2	97.9	31	32.9	47.1	98.4	30.5	31.9	46.6

we conducted a comprehensive analysis of predictive models for software vulnerabilities, exploring various feature set combinations to enhance our understanding of their impact on predictions. Our study highlights the significance of feature set and combinations in predictive modeling for software vulnerabilities. It provides valuable insights into the effectiveness of different feature sets, offering guidance for future model development and emphasizing the need for a nuanced evaluation approach that considers both performance metrics and effect size differences.

APPENDIX

See Tables 9–11.

REFERENCES

- [1] I. Ozkaya, "Can we really achieve software quality?" *IEEE Softw.*, vol. 38, no. 3, pp. 3–6, May 2021.
- [2] W. Cunningham, "The WyCash portfolio management system," *ACM SIGPLAN OOPS Messenger*, vol. 4, no. 2, pp. 29–30, Apr. 1993.
- [3] P. Avgeriou, P. Kruchten, I. Ozkaya, and C. Seaman, "Managing technical debt in software engineering (dagstuhl seminar 16162)," *Dagstuhl Rep.*, vol. 6, no. 4, pp. 110–138, 2016.
- [4] P. Kruchten, R. L. Nord, I. Ozkaya, and J. Visser, "Technical debt in software development: From metaphor to theory," *ACM SIGSOFT Soft. Eng. Notes*, vol. 37, no. 5, pp. 36–38, 2012.
- [5] R. L. Nord, I. Ozkaya, and F. Shull, "Software vulnerabilities, defects, and design flaws: A technical debt perspective," *Tech. Rep.*, 2017.
- [6] *ISO/IEC 25010:2011*. Accessed: Nov. 2022. [Online]. Available: <http://www.iso.org/standard/>
- [7] G. McGraw, "Software security," *IEEE Secur. Privacy*, vol. 2, no. 2, pp. 80–83, Feb. 2004.
- [8] G. McGraw, *Software Security: Building Security in*, vol. 1. Reading, MA, USA: Addison-Wesley, 2006.
- [9] G. McGraw, "Automated code review tools for security," *IEEE Comput.*, vol. 41, no. 12, pp. 108–111, Dec. 2008.
- [10] S. Harris and F. Maymi, *All-in-One CISSP Exam Guide*. New York, NY, USA: McGraw-Hill, 2016.
- [11] D. Verdon and G. McGraw, "Risk analysis in software design," *IEEE Secur. Privacy*, vol. 2, no. 4, pp. 79–84, Jul. 2004.
- [12] *Common Weakness Enumeration*. Accessed: Nov. 2022. [Online]. Available: http://www.cwe.mitre.org/top25/archive/2019/2019_cwe_top25.html
- [13] I. Ahmed, C. Brindescu, U. A. Mannan, C. Jensen, and A. Sarma, "An empirical examination of the relationship between code smells and merge conflicts," in *Proc. ACM/IEEE Int. Symp. Empirical Softw. Eng. Meas. (ESEM)*, Nov. 2017, pp. 58–67.
- [14] A. A. Elkhail and T. Cerny, "On relating code smells to security vulnerabilities," in *Proc. IEEE IEEE 5th Intl. Conf. Big Data Secur. Cloud (BigDataSecurity) Intl. Conf. High Perform. Smart Comput., (HPSec) IEEE Intl. Conf. Intell. Data Secur. (IDS)*, May 2019, pp. 7–12.
- [15] K. Z. Sultana, Z. Codabux, and B. Williams, "Examining the relationship of code and architectural smells with software vulnerabilities," in *Proc. 27th Asia Pacific Softw. Eng. Conf. (APSEC)*, Dec. 2020, pp. 31–40.
- [16] S. J. Oishwee, Z. Codabux, and N. Stakhanova, "An exploratory study on the relationship of smells and design issues with software vulnerabilities," in *Proc. 1st Int. Workshop Mining Softw. Repositories Appl. Privacy Secur.*, Nov. 2022, pp. 16–20.
- [17] M. Siavvas, D. Tsoukalas, M. Jankovic, D. Kehagias, and D. Tzovaras, "Technical debt as an indicator of software security risk: A machine learning approach for software development enterprises," *Enterprise Inf. Syst.*, vol. 16, no. 5, May 2022, Art. no. 1824017.
- [18] H. Mumtaz, M. Alshayeb, S. Mahmood, and M. Niazi, "An empirical study to improve software security through the application of code refactoring," *Inf. Softw. Technol.*, vol. 96, pp. 112–125, Apr. 2018.
- [19] M. Siavvas, D. Tsoukalas, M. Janković, D. Kehagias, A. Chatzigeorgiou, D. Tzovaras, N. Aničić, and E. Gelenbe, "An empirical evaluation of the relationship between technical debt and software security," in *Proc. 9th Int. Conf. Inf. Soc. Technol.*, Mar. 2019.

- [20] A. Gupta, D. Sharma, and K. Phulli, "ANN modelling on vulnerabilities detection in code smells-associated Android applications," *Found. Comput. Decis. Sci.*, vol. 47, no. 1, pp. 3–26, Feb. 2022.
- [21] R. L. Nord, İ. Özkaya, E. J. Schwartz, F. Shull, and R. Kazman, "Can knowledge of technical debt help identify software vulnerabilities," in *Proc. 9th Workshop Cyber Secur. Experimentation Test*, 2016.
- [22] K. Rindell, K. Bernsmed, and M. G. Jaatun, "Managing security in software: Or: How I learned to stop worrying and manage the security technical debt," in *Proc. 14th Int. Conf. Availability, Rel. Secur.*, Aug. 2019, pp. 1–8.
- [23] K. Rindell and J. Holvitie, "Security risk assessment and management as technical debt," in *Proc. Int. Conf. Cyber Secur. Protection Digit. Services (Cyber Security)*, Jun. 2019, pp. 1–8.
- [24] Y. Shin and L. Williams, "An empirical model to predict security vulnerabilities using code complexity metrics," in *Proc. 2nd ACM-IEEE Int. Symp. Empirical Softw. Eng. Meas.*, Oct. 2008, pp. 315–317.
- [25] M. Siavvas, D. Kehagias, and D. Tzovaras, "A preliminary study on the relationship among software metrics and specific vulnerability types," in *Proc. Int. Conf. Comput. Sci. Comput. Intell. (CSCI)*, Dec. 2017, pp. 916–921.
- [26] T. Coskun, R. Halepmollasi, K. Hanifi, R. F. Fouladi, P. C. De Cnudde, and A. Tosun, "Profiling developers to predict vulnerable code changes," in *Proc. 18th Int. Conf. Predictive Models Data Anal. Softw. Eng.*, Nov. 2022, pp. 32–41.
- [27] R. Halepmollasi, K. Hanifi, R. Fouladi, and A. Tosun, "A comparison of source code representation methods to predict vulnerability inducing code changes," in *Proc. 18th Int. Conf. Eval. Novel Approaches to Softw. Eng.*, 2023, pp. 469–478.
- [28] L. Bao, X. Xia, A. E. Hassan, and X. Yang, "V-SZZ: Automatic identification of version ranges affected by CVE vulnerabilities," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng. (ICSE)*, May 2022, pp. 2352–2364.
- [29] *Static Code Analyzer, PMD*. Accessed: May 2023. [Online]. Available: <https://pmd.github.io>
- [30] Z. Li, P. Avgeriou, and P. Liang, "A systematic mapping study on technical debt and its management," *J. Syst. Softw.*, vol. 101, pp. 193–220, Mar. 2015.
- [31] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Cham, Switzerland: Springer, 2012.
- [32] R. Alfayez, R. Winn, Y. Ding, G. Alfayez, and B. Boehm, "Technical debt (TD) through the lens of Twitter: A survey," *J. Softw., Evol. Process*, vol. 36, no. 4, Apr. 2024, Art. no. e2547.
- [33] R. Alfayez, Y. Ding, R. Winn, G. Alfayez, C. Harman, and B. Boehm, "What is asked about technical debt (TD) on stack exchange question-and-answer (Q&A) websites? An observational study," *Empirical Softw. Eng.*, vol. 28, no. 2, p. 35, Mar. 2023.
- [34] P. Kruchten, R. L. Nord, I. Ozkaya, and D. Falessi, "Technical debt: Towards a crisper definition," in *Proc. 4th int. Workshop Manag. Tech. Debt*, Jun. 2013, vol. 38, no. 5, pp. 51–54.
- [35] S. Bellomo, R. L. Nord, I. Ozkaya, and M. Popeck, "Got technical debt? Surfacing elusive technical debt in issue trackers," in *Proc. IEEE/ACM 13th Work. Conf. Mining Softw. Repositories (MSR)*, May 2016, pp. 327–338.
- [36] G. Bavota and B. Russo, "A large-scale empirical study on self-admitted technical debt," in *Proc. IEEE/ACM 13th Work. Conf. Mining Softw. Repositories (MSR)*, May 2016, pp. 315–326.
- [37] I. Ozkaya, Z. Kurtz, R. L. Nord, R. S. Sangwan, and S. M. Srinivasan, "Detecting discussions of technical debt," 2022, *arXiv:2201.12177*.
- [38] P. Avgeriou, P. Kruchten, R. L. Nord, I. Ozkaya, and C. Seaman, "Reducing friction in software development," *IEEE Softw.*, vol. 33, no. 1, pp. 66–73, Jan. 2016.
- [39] B. Paudel, J. Gonzalez-Huerta, E. Zabardast, and E. Klotins, "Towards measuring the impact of technical debt on lead time: An industrial case study," 2024, *arXiv:2406.01578*.
- [40] I. Gat and J. D. Heintz, "From assessment to reduction: How cutter consortium helps rein in millions of dollars in technical debt," in *Proc. 2nd Workshop Manag. Tech. Debt*, Israel, May 2011, pp. 24–26.
- [41] A. Nugroho, J. Visser, and T. Kuipers, "An empirical model of technical debt and interest," in *Proc. 2nd Workshop Manag. Tech. Debt*, May 2011, pp. 1–8.
- [42] B. Curtis, J. Sappidi, and A. Szykarski, "Estimating the size, cost, and types of technical debt," in *Proc. 3rd Int. Workshop Manag. Tech. Debt (MTD)*, Jun. 2012, pp. 49–53.
- [43] *A Product for Clean Code, Sonarqube*. Accessed: Feb. 2023. [Online]. Available: <http://www.sonarqube.org>
- [44] *Augmented Software Analytics, Squire*. Accessed: Feb. 2023. [Online]. Available: <https://www.vector.com/int/en/products/products-a-z/software/squire/>
- [45] *Code Security, Kiuwan*. Accessed: Feb. 2023. [Online]. Available: <https://www.kiuwan.com>
- [46] R. Marinescu, "Assessing technical debt by identifying design flaws in software systems," *IBM J. Res. Develop.*, vol. 56, no. 5, pp. 1–13, Sep. 2012.
- [47] Y. Cai, R. Kazman, C. V. A. Silva, L. Xiao, and H.-M. Chen, "A decision-support system approach to economics-driven modularity evaluation," in *Economics-Driven Software Architecture*. Amsterdam, The Netherlands: Elsevier, 2014, pp. 105–128.
- [48] F. A. Fontana, V. Ferme, and S. Spinelli, "Investigating the impact of code smells debt on quality code evaluation," in *Proc. 3rd Int. Workshop Manag. Tech. Debt (MTD)*, Jun. 2012, pp. 15–22.
- [49] M. Fowler, *Refactoring: Improving the Design of Existing Code*. Reading, MA, USA: Addison-Wesley, 2018.
- [50] L. Xiao, Y. Cai, R. Kazman, R. Mo, and Q. Feng, "Identifying and quantifying architectural debt," in *Proc. IEEE/ACM 38th Int. Conf. Softw. Eng. (ICSE)*, May 2016, pp. 488–498.
- [51] B. Du Bois, S. Demeyer, and J. Verelst, "Refactoring—Improving coupling and cohesion of existing code," in *Proc. 11th Work. Conf. Reverse Eng.*, Mar. 2004, pp. 144–151.
- [52] S. Wong, Y. Cai, M. Kim, and M. Dalton, "Detecting software modularity violations," in *Proc. 33rd Int. Conf. Softw. Eng. (ICSE)*, May 2011, pp. 411–420.
- [53] B. Pérez, C. Castellanos, D. Correal, N. Rios, S. Freire, R. Spínola, and C. Seaman, "What are the practices used by software practitioners on technical debt payment: Results from an international family of surveys," in *Proc. 3rd Int. Conf. Tech. Debt*, Jun. 2020, pp. 103–112.
- [54] B. Pérez, C. Castellanos, D. Correal, N. Rios, S. Freire, R. Spínola, C. Seaman, and C. Izurieta, "Technical debt payment and prevention through the lenses of software architects," *Inf. Softw. Technol.*, vol. 140, Dec. 2021, Art. no. 106692.
- [55] G. Lacerda, F. Petrillo, M. Pimenta, and Y. G. Guéhenneuc, "Code smells and refactoring: A tertiary systematic review of challenges and observations," *J. Syst. Softw.*, vol. 167, Sep. 2020, Art. no. 110610.
- [56] G. Bavota, A. De Lucia, M. Di Penta, R. Oliveto, and F. Palomba, "An experimental investigation on the innate relationship between quality and refactoring," *J. Syst. Softw.*, vol. 107, pp. 1–14, Sep. 2015.
- [57] E. A. AlOmar, M. W. Mkaouer, C. Newman, and A. Ouni, "On preserving the behavior in software refactoring: A systematic mapping study," *Inf. Softw. Technol.*, vol. 140, Dec. 2021, Art. no. 106675.
- [58] M. BenIdris, H. Ammar, and D. Dzielski, "The technical debt density over multiple releases and the refactoring story," *Int. J. Softw. Eng. Knowl. Eng.*, vol. 31, no. 1, pp. 99–116, Jan. 2021.
- [59] E. Murphy-Hill, C. Parnin, and A. P. Black, "How we refactor, and how we know it," *IEEE Trans. Softw. Eng.*, vol. 38, no. 1, pp. 5–18, Jan. 2012.
- [60] A. Peruma, M. W. Mkaouer, M. J. Decker, and C. D. Newman, "An empirical investigation of how and why developers rename identifiers," in *Proc. 2nd Int. Workshop Refactoring*, Peru, Sep. 2018, pp. 26–33.
- [61] D. Silva, N. Tsantalis, and M. T. Valente, "Why we refactor? Confessions of GitHub contributors," in *Proc. 24th ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, Nov. 2016, pp. 858–870.
- [62] M. Alshayeb, "Empirical investigation of refactoring effect on software quality," *Inf. Softw. Technol.*, vol. 51, no. 9, pp. 1319–1326, Sep. 2009.
- [63] A. Chávez, I. Ferreira, E. Fernandes, D. Cedrim, and A. Garcia, "How does refactoring affect internal quality attributes? A multi-project study," in *Proc. 31st Brazilian Symp. Softw. Eng.*, Apr. 2017, pp. 74–83.
- [64] R. Moser, P. Abrahamsson, W. Pedrycz, A. Sillitti, and G. Succi, "A case study on the impact of refactoring on quality and productivity in an agile team," in *Proc. Balancing Agility Formalism Softw. Eng., 2nd IFIP TC 2 Central East Eur. Conf. Softw. Eng. Techn.*, Poznan, Poland. Cham, Switzerland: Springer, Oct. 2008, pp. 252–266.
- [65] E. AlOmar, M. W. Mkaouer, and A. Ouni, "Can refactoring be self-affirmed? An exploratory study on how developers document their refactoring activities in commit messages," in *Proc. IEEE/ACM 3rd Int. Workshop Refactoring (IWorR)*, May 2019, pp. 51–58.
- [66] E. A. Alomar, A. Peruma, M. W. Mkaouer, C. D. Newman, and A. Ouni, "Behind the scenes: On the relationship between developer experience and refactoring," *J. Softw., Evol. Process*, vol. 36, no. 1, Jan. 2024.

- [67] D. Cedrim, A. Garcia, M. Mongiovi, R. Gheyi, L. Sousa, R. de Mello, B. Fonseca, M. Ribeiro, and A. Chávez, "Understanding the impact of refactoring on smells: A longitudinal study of 23 software projects," in *Proc. 11th Joint Meeting Found. Softw. Eng.*, Aug. 2017, pp. 465–475.
- [68] R. Halepmollasi and A. Tosun, "Exploring the relationship between refactoring and code debt indicators," *J. Softw., Evol. Process.*, vol. 36, no. 1, Jan. 2024, Art. no. e2447.
- [69] A. Gautier, C. Whitehead, D. Dzielski, T. Devine, and J. Hernandez, "A systematic mapping study of the advancement in software vulnerability forecasting," in *Proc. SoutheastCon*, Apr. 2023, pp. 545–552.
- [70] R. Wang, S. Xu, X. Ji, Y. Tian, L. Gong, and K. Wang, "An extensive study of the effects of different deep learning models on code vulnerability detection in Python code," *Automated Softw. Eng.*, vol. 31, no. 1, p. 15, May 2024.
- [71] Z. Chu, Y. Wan, Q. Li, Y. Wu, H. Zhang, Y. Sui, G. Xu, and H. Jin, "Graph neural networks for vulnerability detection: A counterfactual explanation," in *Proc. 33rd ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Sep. 2024, pp. 389–401.
- [72] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, Mar. 2019.
- [73] V. H. Nguyen and L. M. S. Tran, "Predicting vulnerable software components with dependency graphs," in *Proc. 6th Int. Workshop Secur. Meas. Metrics*, Sep. 2010, pp. 1–8.
- [74] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *Proc. IEEE Symp. Secur. Privacy*, May 2014, pp. 590–604.
- [75] F. Yamaguchi, M. Lottmann, and K. Rieck, "Generalized vulnerability extrapolation using abstract syntax trees," in *Proc. 28th Annu. Comput. Secur. Appl. Conf.*, Dec. 2012, pp. 359–368.
- [76] F. Yamaguchi, C. Wressnegger, H. Gascon, and K. Rieck, "Chucky: Exposing missing checks in source code for vulnerability discovery," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CSS)*, 2013, pp. 499–510.
- [77] F. Yamaguchi, "Pattern-based vulnerability discovery," Tech. Rep., 2015.
- [78] S. E. Şahin, E. M. Özyedierler, and A. Tosun, "Predicting vulnerability inducing function versions using node embeddings and graph neural networks," *Inf. Softw. Technol.*, vol. 145, May 2022, Art. no. 106822.
- [79] T. H. M. Le, M. A. Babar, and T. H. Thai, "Software vulnerability prediction in low-resource languages: An empirical study of CodeBERT and ChatGPT," in *Proc. 28th Int. Conf. Eval. Assessment Softw. Eng.*, Jun. 2024, pp. 679–685.
- [80] G. Lu, X. Ju, X. Chen, W. Pei, and Z. Cai, "GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning," *J. Syst. Softw.*, vol. 212, Jun. 2024, Art. no. 112031.
- [81] M. C. O. Silva, M. T. Valente, and R. Terra, "Does technical debt lead to the rejection of pull requests?" 2016, *arXiv:1604.01450*.
- [82] C. Izurieta, K. Kimball, D. Rice, and T. Valentien, "A position study to investigate technical debt associated with security weaknesses," in *Proc. IEEE/ACM Int. Conf. Tech. Debt (TechDebt)*, May 2018, pp. 138–142.
- [83] C. Izurieta and M. Prouty, "Leveraging SecDevOps to tackle the technical debt associated with cybersecurity attack tactics," in *Proc. IEEE/ACM Int. Conf. Tech. Debt (TechDebt)*, May 2019, pp. 33–37.
- [84] R. Kazman, Y. Cai, R. Mo, Q. Feng, L. Xiao, S. Haziye, V. Fedak, and A. Shapochka, "A case study in locating the architectural roots of technical debt," in *Proc. IEEE/ACM 37th IEEE Int. Conf. Softw. Eng.*, vol. 2, May 2015, pp. 179–188.
- [85] A. Zheng and A. Casari, "Feature engineering for machine learning: Principles and techniques for data scientists," O'Reilly Media, 2018.
- [86] R. Croft, Y. Xie, and M. A. Babar, "Data preparation for software vulnerability prediction: A systematic literature review," *IEEE Trans. Softw. Eng.*, vol. 49, no. 3, pp. 1044–1063, Mar. 2023.
- [87] A. Trautsch, F. Trautsch, and S. Herbold, "MSR mining challenge: The SmartSHARK repository mining data," 2021, *arXiv:2102.11540*.
- [88] Z. Song, Y. Wang, P. Zong, Z. Ren, and D. Qi, "An empirical study of comparison of code metric aggregation methods—On embedded software," in *Proc. IEEE 19th Int. Conf. Softw. Qual., Rel. Secur. Companion (QRS-C)*, Jul. 2019, pp. 114–119.
- [89] N. Tsantalis, M. Mansouri, D. Eshkevari, D. Mazinanian, and D. Dig, "Accurate and efficient refactoring detection in commit history," in *Proc. IEEE/ACM 40th Int. Conf. Softw. Eng. (ICSE)*, May 2018, pp. 483–494.
- [90] Z. Xing and E. Stroulia, "UMLDiff: An algorithm for object-oriented design differencing," in *Proc. 20th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Nov. 2005, pp. 54–65.
- [91] B. Biegel, Q. D. Soetens, W. Hornig, S. Diehl, and S. Demeyer, "Comparison of similarity metrics for refactoring detection," in *Proc. 8th Work. Conf. Mining Softw. Repositories*, May 2011, pp. 53–62.
- [92] H. Keuning, B. Heeren, and J. Jeurig, "Code quality issues in student programs," in *Proc. ACM Conf. Innov. Technol. Comput. Sci. Educ.*, Jun. 2017, pp. 110–115.
- [93] Z. Di, B. Li, Z. Li, and P. Liang, "A preliminary investigation of self-admitted refactorings in open source software (S)," in *Proc. Int. Conf. Softw. Eng. Knowl. Eng.*, vol. 2018, Jul. 2018, pp. 165–168.
- [94] J. Nam, S. Wang, Y. Xi, and L. Tan, "A bug finder refined by a large set of open-source projects," *Inf. Softw. Technol.*, vol. 112, pp. 164–175, Aug. 2019.
- [95] D. Spadini, M. Aniche, and A. Bacchelli, "PyDriller: Python framework for mining software repositories," in *Proc. 26th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Oct. 2018, pp. 908–911.
- [96] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3280–3296, Sep. 2022.
- [97] S. E. Ponta, H. Plate, A. Sabetta, M. Bezzi, and C. Dangremont, "A manually-curated dataset of fixes to vulnerabilities of open-source software," in *Proc. IEEE/ACM 16th Int. Conf. Mining Softw. Repositories (MSR)*, May 2019, pp. 383–387.
- [98] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.
- [99] A. Fernandez, S. Garcia, F. Herrera, and N. V. Chawla, "SMOTE for learning from imbalanced data: Progress and challenges, marking the 15-year anniversary," *J. Artif. Intell. Res.*, vol. 61, pp. 863–905, Apr. 2018.
- [100] J. Bohnet and J. Döllner, "Monitoring code quality and development activity by software maps," in *Proc. 2nd Workshop Manag. Tech. Debt*, May 2011, pp. 9–16.
- [101] B. Barton and C. Sterling, "Manage project portfolios more effectively by including software debt in the decision process," *Cutter IT J.*, vol. 23, no. 10, p. 19, 2010.
- [102] J. Holvitie and V. Leppänen, "DebtFlag: Technical debt management with a development environment integrated tool," in *Proc. 4th Int. Workshop Manag. Tech. Debt (MTD)*, May 2013, pp. 20–27.
- [103] T. S. Mendes, F. G. S. Gomes, D. P. Gonçalves, M. G. Mendonça, R. L. Novais, and R. O. Spínola, "VisminterTD: A tool for automatic identification and interactive monitoring of the evolution of technical debt items," *J. Brazilian Comput. Soc.*, vol. 25, no. 1, pp. 1–28, Dec. 2019.
- [104] M. Fowler, K. Beck, J. Brant, W. Opdyke, and D. Roberts, *Refactoring: Improving the Design of Existing Code*. Reading, MA, USA: Addison-Wesley, 1999.
- [105] N. Szworka, A. Vetro, C. Izurieta, S. Wong, Y. Cai, C. Seaman, and F. Shull, "Comparing four approaches for technical debt identification," *Softw. Quality J.*, vol. 22, no. 3, pp. 403–426, Sep. 2014.
- [106] Z. Codabux and C. Dutchyn, "Profiling developers through the lens of technical debt," in *Proc. 14th ACM / IEEE Int. Symp. Empirical Softw. Eng. Meas. (ESEM)*, Oct. 2020, pp. 1–6.
- [107] A. J. Ko and B. A. Myers, "A framework and methodology for studying the causes of software errors in programming systems," *J. Vis. Lang. Comput.*, vol. 16, nos. 1–2, pp. 41–84, Feb. 2005.
- [108] P. He, B. Li, Y. Ma, and L. He, "Using software dependency to bug prediction," *Math. Problems Eng.*, vol. 2013, pp. 1–12, Jun. 2013.
- [109] S. Zaman, B. Adams, and A. E. Hassan, "A qualitative study on performance bugs," in *Proc. 9th IEEE Work. Conf. Mining Softw. Repositories (MSR)*, Jun. 2012, pp. 199–208.
- [110] E. Zabardast, J. Gonzalez-Huerta, and D. Šmite, "Refactoring, bug fixing, and new development effect on technical debt: An industrial case study," in *Proc. 46th Euromicro Conf. Softw. Eng. Adv. Appl. (SEAA)*, Aug. 2020, pp. 376–384.
- [111] J. Xuan, Y. Hu, and H. Jiang, "Debt-prone bugs: Technical debt in software maintenance," 2017, *arXiv:1704.04766*.
- [112] G. Suryanarayana, G. Samarthyam, and T. Sharma, *Refactoring for Software Design Smells: Managing Technical Debt*. San Mateo, CA, USA: Morgan Kaufmann, 2014.
- [113] R. Halepmollasi and A. Tosun, *Vulnerability_prediction_github*. [Online]. Available: <http://github.com/Rusen-Akkus/t4vulnerability-prediction>

- [114] L. Yu and H. Liu, "Feature selection for high-dimensional data: A fast correlation-based filter solution," in *Proc. 20th Int. Conf. Mach. Learn. (ICML)*, Apr. 2003, pp. 856–863.
- [115] K. I. Diamantaras and S. Y. Kung, *Principal Component Neural Networks: Theory and Applications*. Hoboken, NJ, USA: Wiley, 1996.
- [116] P. Comon, "Independent component analysis, a new concept?" *Signal Process.*, vol. 36, no. 3, pp. 287–314, Apr. 1994.
- [117] J. A. K. Suykens and J. Vandewalle, "Least squares support vector machine classifiers," *Neural Process. Lett.*, vol. 9, no. 3, pp. 293–300, Jun. 1999.
- [118] T. Chen, T. He, M. Benesty, V. Khotilovich, Y. Tang, H. Cho, K. Chen, R. Mitchell, I. Cano, T. Zhou, M. Li, J. Xie, M. Lin, Y. Geng, Y. Li, and J. Yuan, "Xgboost: Extreme gradient boosting," *R Package Version 0.4-2*, vol. 1, no. 4, pp. 1–4, 2014.
- [119] S. Kotsiantis, D. Kanellopoulos, and P. Pintelas, "Handling imbalanced datasets: A review," *GESTS Int. Trans. Comput. Sci. Eng.*, vol. 30, no. 1, pp. 25–36, 2006.
- [120] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, and V. Dubourg, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, May 2011.
- [121] A. Majeed and S. O. Hwang, "Technical analysis of data-centric and model-centric artificial intelligence," *IT Prof.*, vol. 25, no. 6, pp. 62–70, Nov. 2023.
- [122] F. Wu, J. Wang, J. Liu, and W. Wang, "Vulnerability detection with deep learning," in *Proc. 3rd IEEE Int. Conf. Comput. Commun. (ICCC)*, Dec. 2017, pp. 1298–1302.



RUSEN AKKUS HALEPMOLLASI received the bachelor's degree in computer engineering from Mersin University, in 2006, and the M.Sc. and Ph.D. degrees in computer engineering from Istanbul Technical University.

From 2004 to 2007, she was a part-time and full-time Researcher at various global telecommunications and industrial companies. She was a Lecturer at Harran University and Usak University, before joining Istanbul Technical University, as a

Research and Teaching Assistant, from 2011 to 2021. During her time in the Machine Learning Laboratory, she conducted research in bioinformatics, utilizing machine learning, statistical analysis, and data science methodologies,

until 2016. From 2016 to 2021, she was with the Software Modeling Laboratory, focusing on software engineering, software security, software quality, the software development lifecycle, and agile methodologies. In collaboration with Ericsson, she conducted research on predicting vulnerabilities in software systems. She joined TÜBİTAK BİLGEM Information Technologies Institute, as a Senior Researcher, in 2021. She was the Technical Leader on various projects and, since 2023, she has been the Head of the Cloud Computing and Big Data Research Laboratory. In this role, she manages multiple development teams comprising software engineers, data scientists, data engineers, and systems engineers. She is responsible for carrying out international and national project cooperation, project preparation, and management processes on machine learning, big data, cloud computing, federated learning, edge computing, and digital twins.



AYSE TOSUN received the M.Sc. and Ph.D. degrees from the Department of Computer Engineering, Boğaziçi University, Türkiye, in 2008 and 2012, respectively.

She is currently an Associate Professor with the Faculty of Computer and Informatics Engineering, Istanbul Technical University (ITU). She was the Vice Dean, the Vice Chair, faculty, and institution executive board member with ITU, since she joined ITU in 2014. Prior to joining ITU, she

was a Postdoctoral Research Fellow with the Department of Information Processing Science, University of Oulu, Finland. Her broad research areas are software engineering and AI. More specifically, she has been leading research projects (industry- and academy-funded), guiding M.Sc. and Ph.D. theses in the field of empirical software engineering with a special focus on mining large volumes of software data through software repositories, analyzing patterns in software data, and building AI-based recommender systems for software engineers. Her recent works concentrate on understanding the reasons behind software vulnerabilities in terms of coding and design debt and improving correctness and fairness of AI-based systems.

Dr. Tosun is on the review boards of the top-tier journals in software engineering, such as IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, *Empirical Software Engineering*, *Information and Software Technology*, *Software Quality Journal*, and *Journal of Systems and Software*. She has been in the program and organization committees of the international conferences on software engineering.

...